SLIIT UNI

THE KNOWLEDGE UNIVERSITY

08 – Application Layer

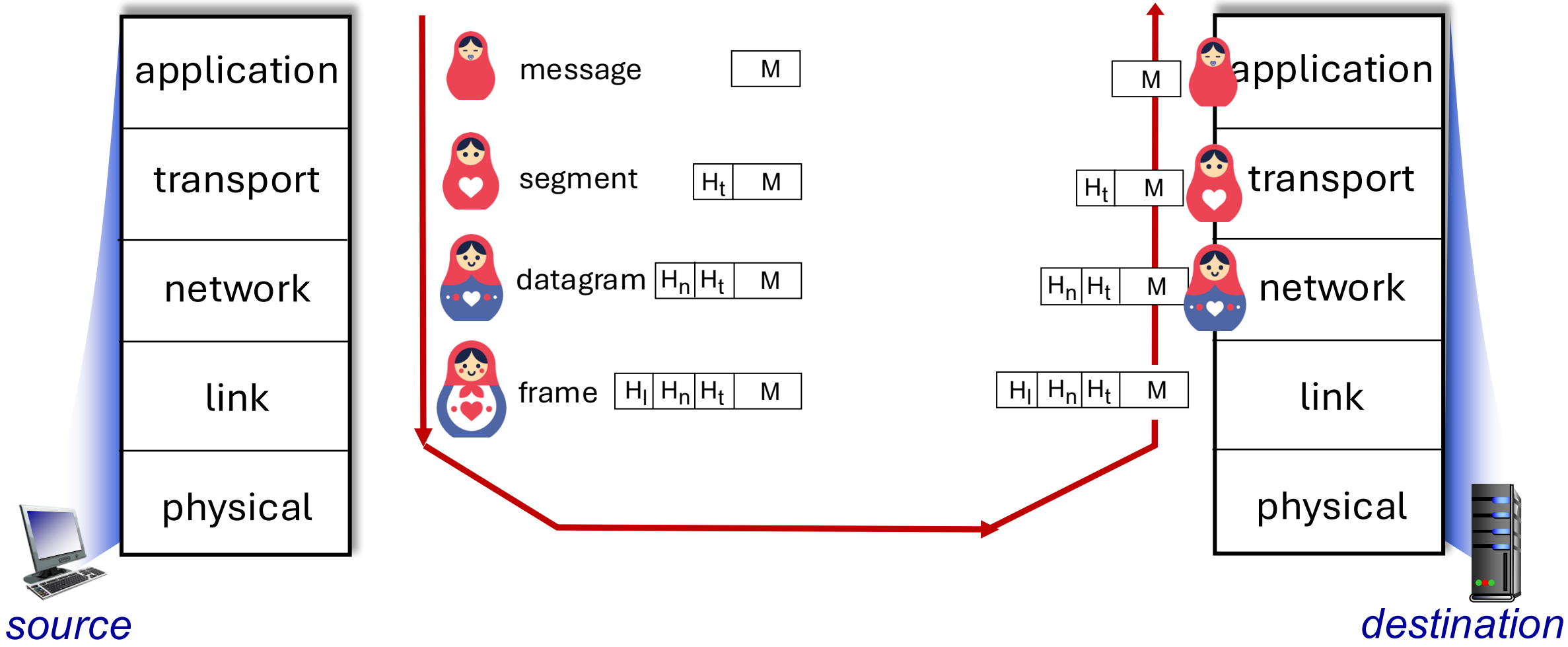
IE2004 Computer Networks

2026 July

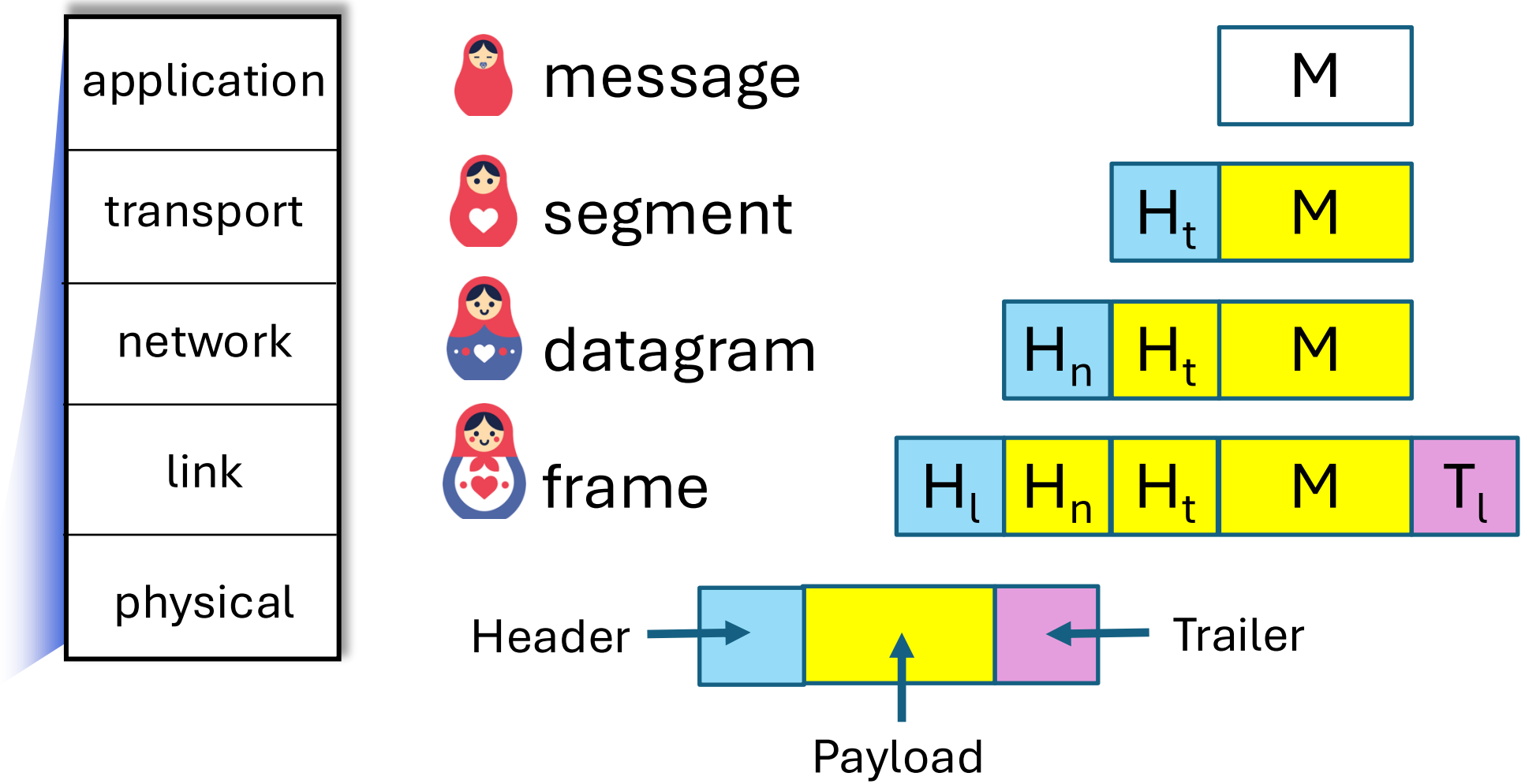
Lesson outline

- Internet basics
- Application layer: overview
- DNS
- HTTP

Services, Layering and Encapsulation



Packets: Headers, Trailers and Payloads



Network layer / Internet Protocol

- Each host on the Internet is identified by its IP address
  - Must be unique on the Internet
- If a host / router has multiple interfaces, a separate IP address is needed for each interface
- IPv4
  - 32 bit addresses: e.g. 11000001 00100000 11011000 00001001
  - dotted decimal notation: each byte (octet) expresses as a decimal number – easier for humans
  - So, the address 193.32.216.9 in binary notation is

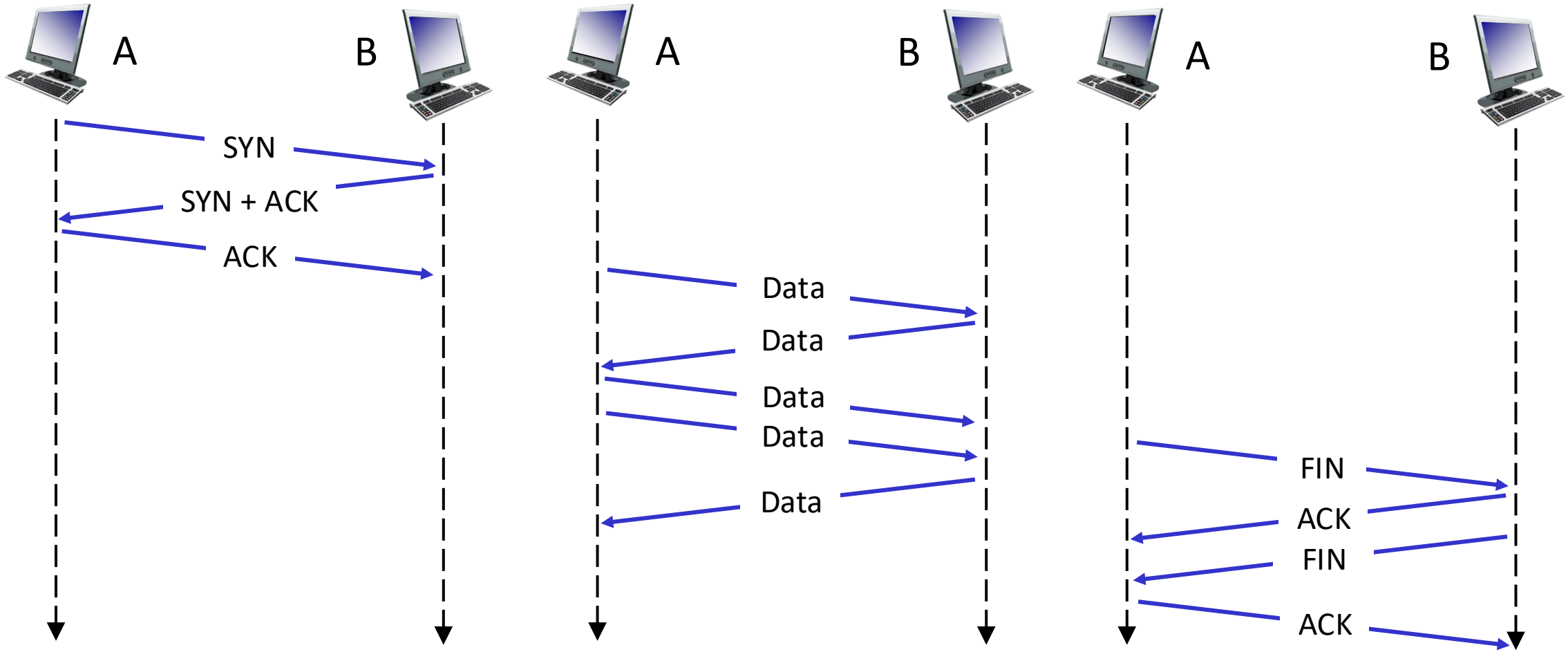
    11000001 00100000 11011000 00001001
- IPv6
  - 128 bit addresses
  - Expressed in colon hexadecimal notation
  - e.g. 2001:0db8:1001:2002:3008:4800:200c:417a

Transport layer / TCP and UDP

- Datagram delivery system in the Network Layer (L3 or IP layer) is
  - Unreliable: Delivery is not guaranteed
  - Best effort: Some datagrams may be dropped due to congestion etc
  - Connectionless: Each datagram is treated independently from others
- What if applications needs reliability?
  - Use a reliable protocol at the transport layer
  - Provide reliability at the application layer protocol
- Transport layer provides two protocols
  - TCP: Connection oriented, Reliable (does extra work to provide reliability)
  - UDP: Connectionless, Same unreliability as IP (doesn't do any extra work)
- As there may be multiple applications running on a single host, we need a way to identify each application / reach them over Internet
  - Just the IP address is not sufficient – common to all applications
  - Transport layer identifies each app using a Port number (16-bit number)

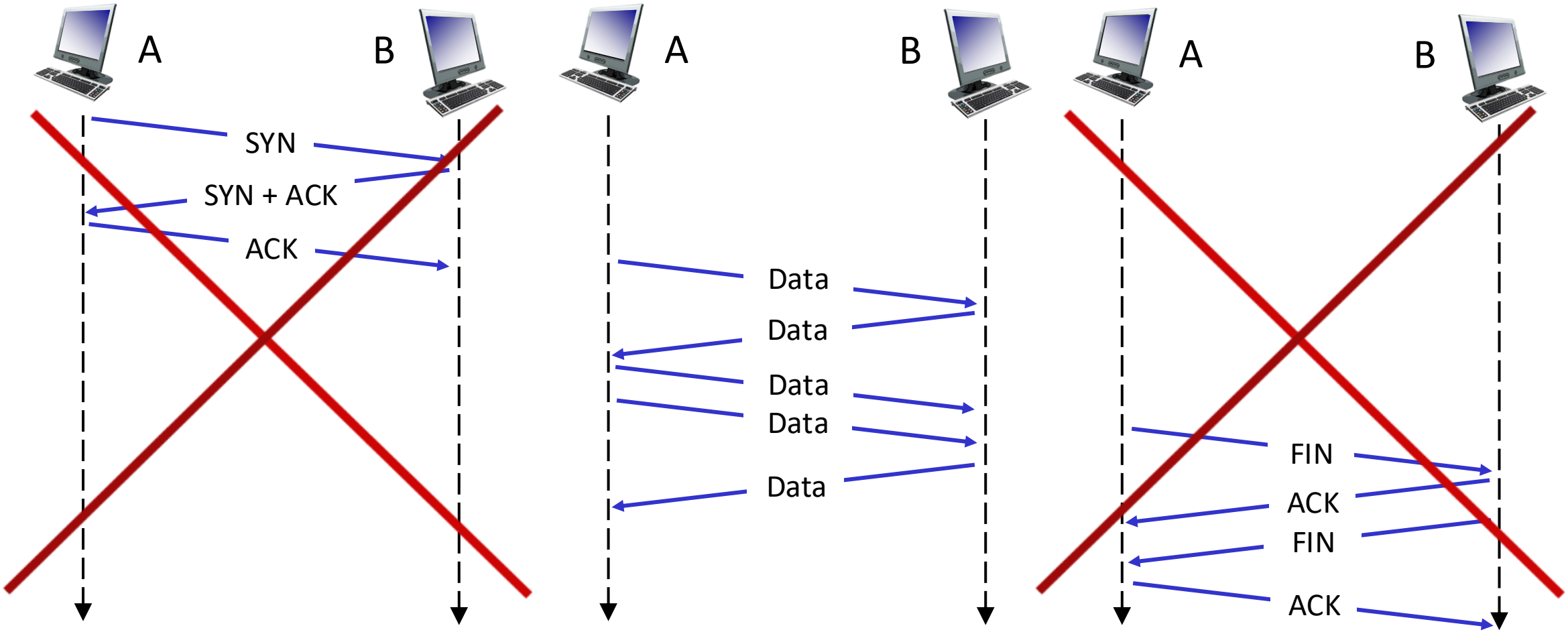
TCP: Connection-oriented, Reliable

- Connection establishment, Data transfer, Connection closure

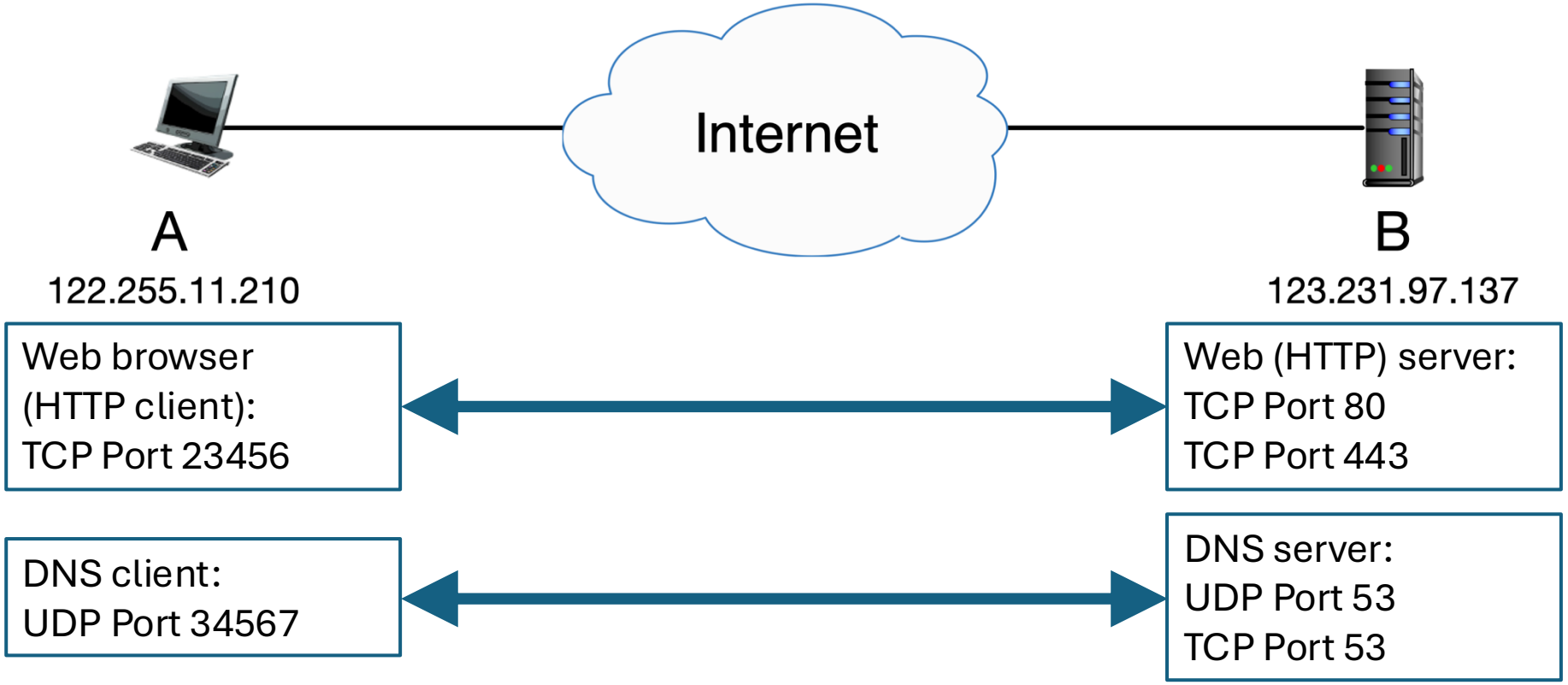


UDP: Connectionless, Same unreliability as IP

- No connection establishment or Connection closure



Applications and Port numbers



- Each application can be uniquely identified & reached over network
  - Web browser is 122.255.11.210:23456 (TCP)
  - DNS client is 122.255.11.210:34567 (UDP)
  - Web server is 123.231.97.137:80 (TCP) and 123.231.97.137:443 (TCP)
  - DNS server is 123.231.97.137:53 (TCP) and 123.231.97.137:53 (UDP)

Lesson outline

- Internet basics
- Application layer: overview
- DNS
- HTTP

Application layer: overview

Our goals:

- conceptual *and* implementation aspects of application-layer protocols
  - transport-layer service models
  - client-server paradigm
  - peer-to-peer paradigm
- learn about protocols by examining popular application-layer protocols and infrastructure
  - DNS
  - HTTP
- programming network applications
  - socket API

Some network apps

- social networking
- AI chatbots
- web browsing
- text messaging
- e-mail
- multi-user network games
- streaming stored video (YouTube, Hulu, Netflix)
- P2P file sharing
- voice over IP (WhatsApp, Enterprise IP phone systems)
- real-time video conferencing (e.g., Zoom)
- Internet search
- remote login (ssh)
- …
- …

• *Q: your* favorites?

Client-server paradigm

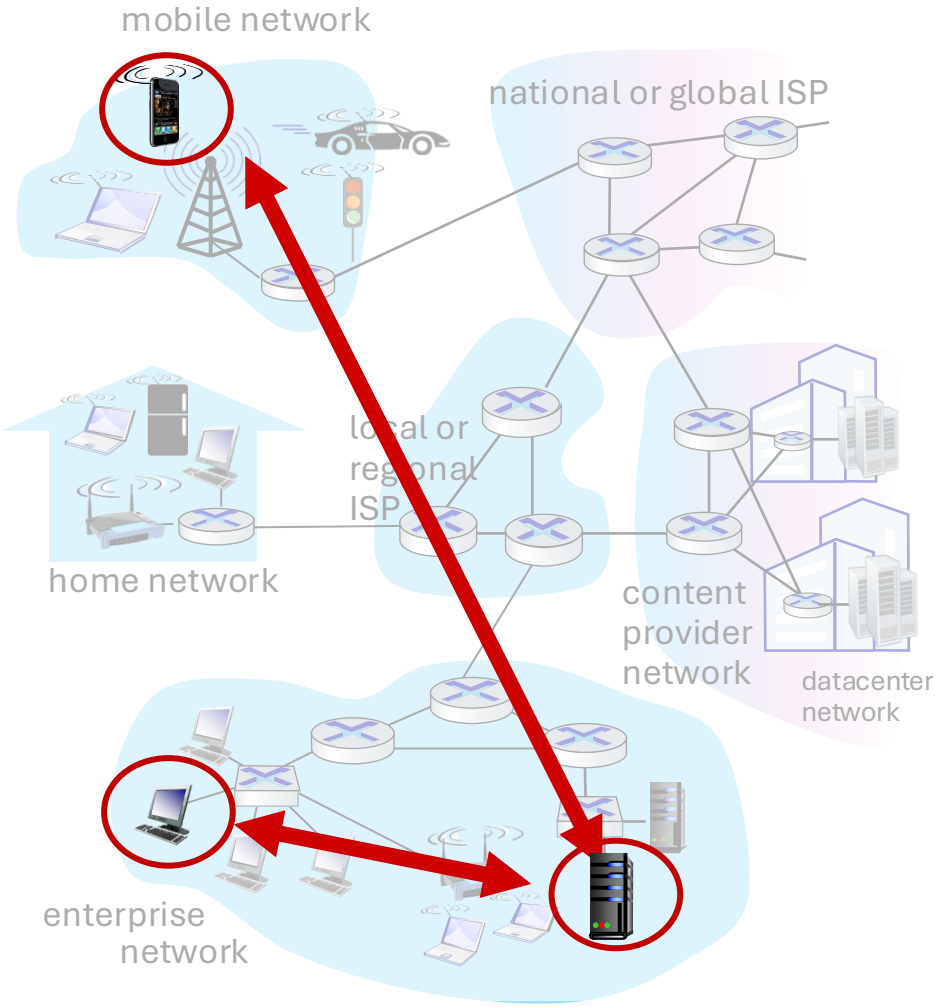
server:

- always-on host
- permanent (static) IP address
- often in data centers, for scaling

clients:

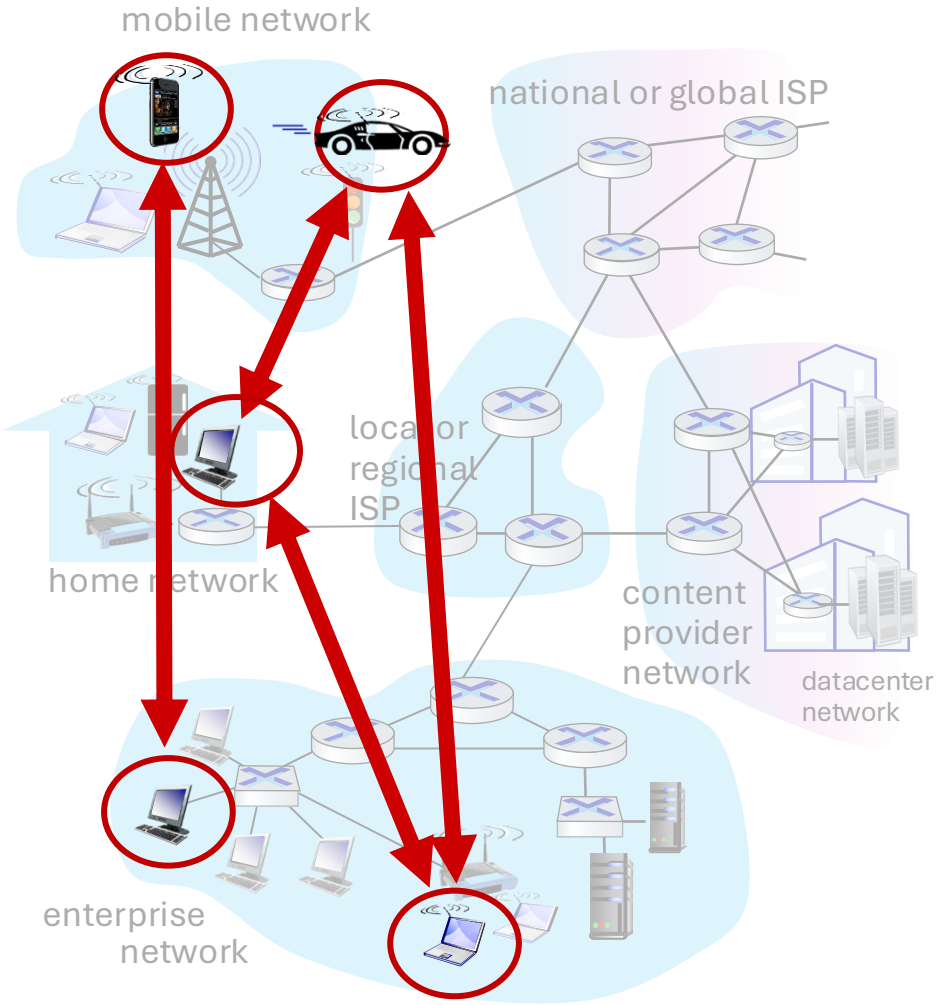
- contact, communicate with server
- may be intermittently connected
- may have dynamic IP addresses
- do *not* communicate directly with each other

- examples: HTTP, IMAP, FTP



Peer-peer architecture

- *no* always-on server
- arbitrary end systems directly communicate
- peers request service from other peers, provide service in return to other peers
  - *self scalability* – new peers bring new service capacity, as well as new service demands
- peers are intermittently connected and change IP addresses
  - complex management
- example: P2P file sharing [BitTorrent]



An application-layer protocol defines:

- types of messages exchanged
  - e.g., request, response
- message syntax
  - what fields are in messages, field width (no of bits in each field), and the order of fields
- message semantics
  - meaning of information in fields
- rules for when and how applications send messages and respond to messages

open protocols:

- defined in RFCs, everyone has access to protocol definition
- allows for interoperability
- e.g., HTTP, SMTP

proprietary protocols:

- e.g., Skype, Zoom

What transport service does an app need?

data integrity

- some apps (e.g., file transfer, web transactions) require 100% reliable data transfer
- other apps (e.g., audio) can tolerate some loss

timing

- some apps (e.g., Internet telephony, interactive games) require low delay to be "effective"

throughput

- some apps (e.g., multimedia) require minimum amount of throughput to be "effective"
- other apps ("elastic apps") make use of whatever throughput they get

security

encryption, data integrity, …

Transport service requirements: common apps

application	data loss	throughput	time sensitive?
file transfer/download	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5Kbps-1Mbps video:10Kbps-5Mbps	yes, 10's msec
streaming audio/video	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	Kbps+	yes, 10's msec
text messaging	no loss	elastic	yes and no

Internet applications, and transport protocols

application	application layer protocol	transport protocol
file transfer/download	FTP [RFC 959]	TCP
e-mail	SMTP [RFC 5321]	TCP
Web documents	HTTP [RFC 7230, 9110]	TCP
Internet telephony	SIP [RFC 3261], RTP [RFC 3550], or	TCP or UDP
streaming audio/video	proprietary HTTP [RFC	TCP
interactive games	7230], DASH	UDP or TCP
	WOW, FPS (proprietary)	

Lesson outline

- Internet basics
- Application layer: overview
- DNS
- HTTP

DNS: Domain Name System

people: many identifiers:

- NIC, name, passport #

Internet hosts, routers:

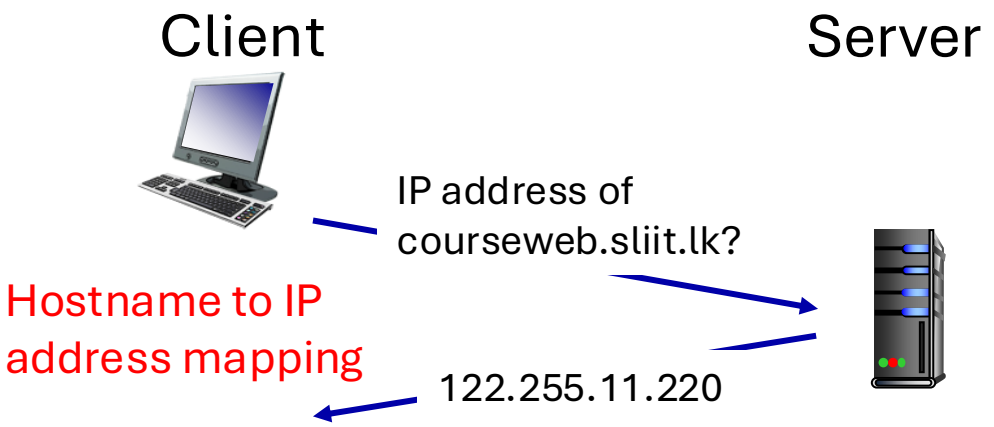
- IP address (32 bit) - used for addressing datagrams
- "name", e.g., www.sliit.lk - used by humans

Q: how to map between IP address and name, and vice versa ?

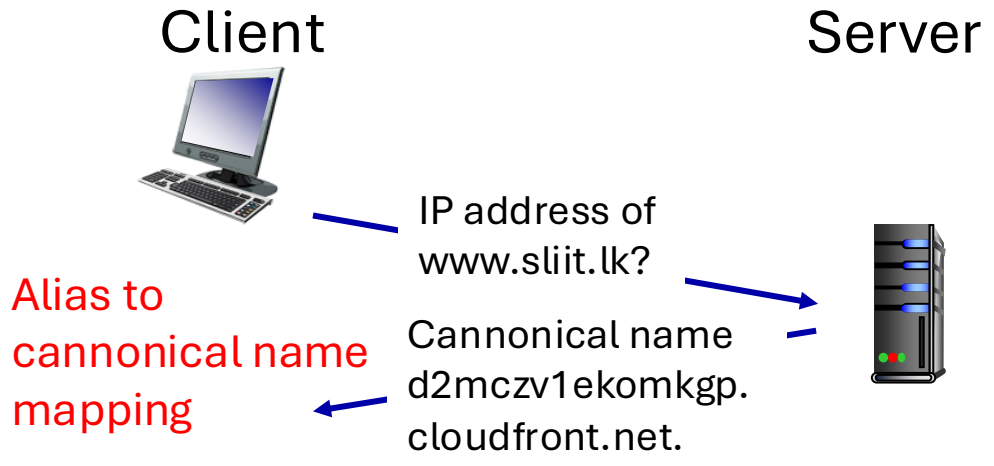
Domain Name System (DNS):

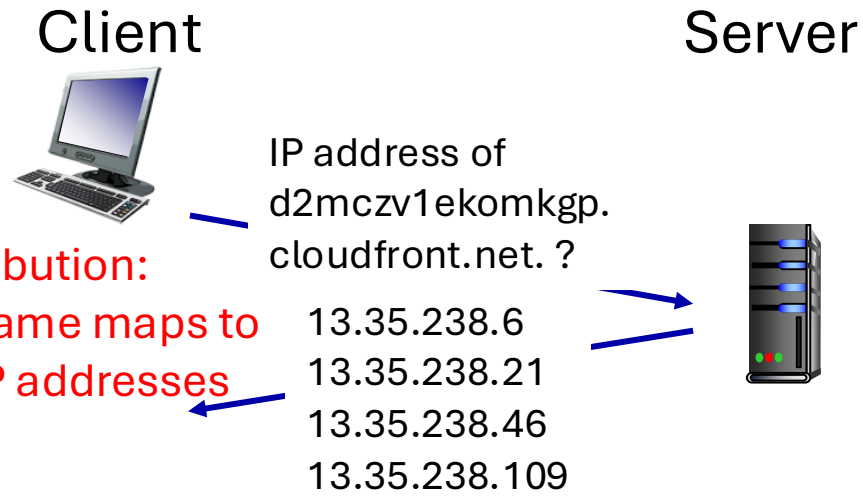
- *distributed database* containing resource records (RRs), implemented in a hierarchy of many *name servers*
- *application-layer protocol:* hosts (clients), name servers communicate to *resolve* names (address/name translation)
- core Internet function, implemented as an application-layer protocol
- DNS server = name server

DNS: requests and responses







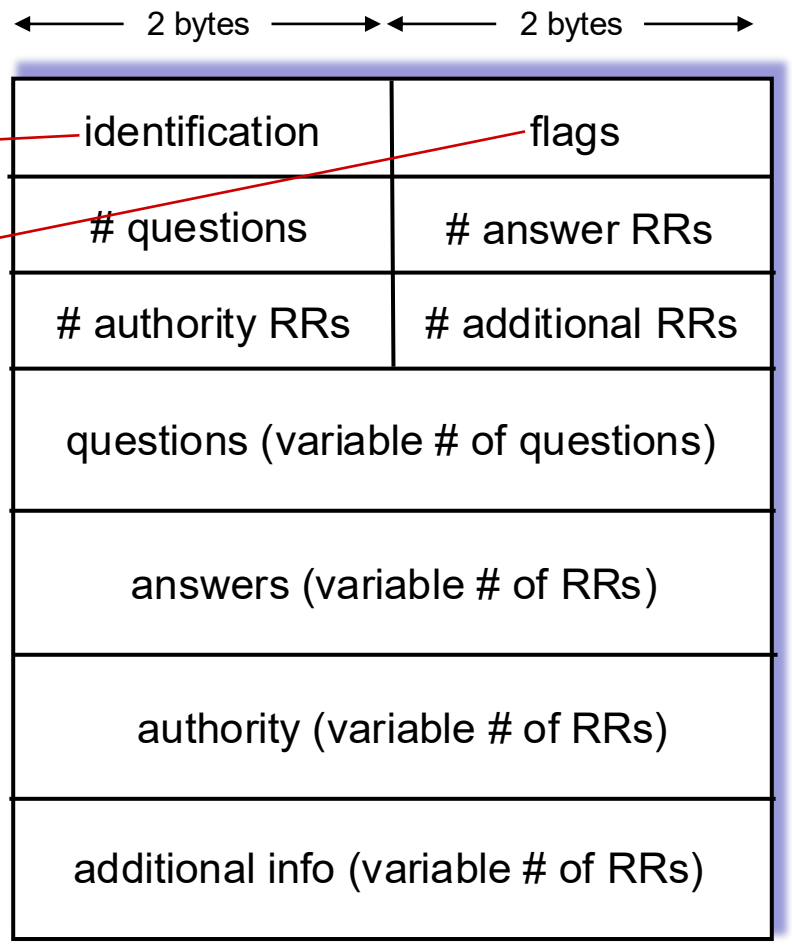


DNS protocol messages

- DNS *query* and *reply* messages, both have same *format:*

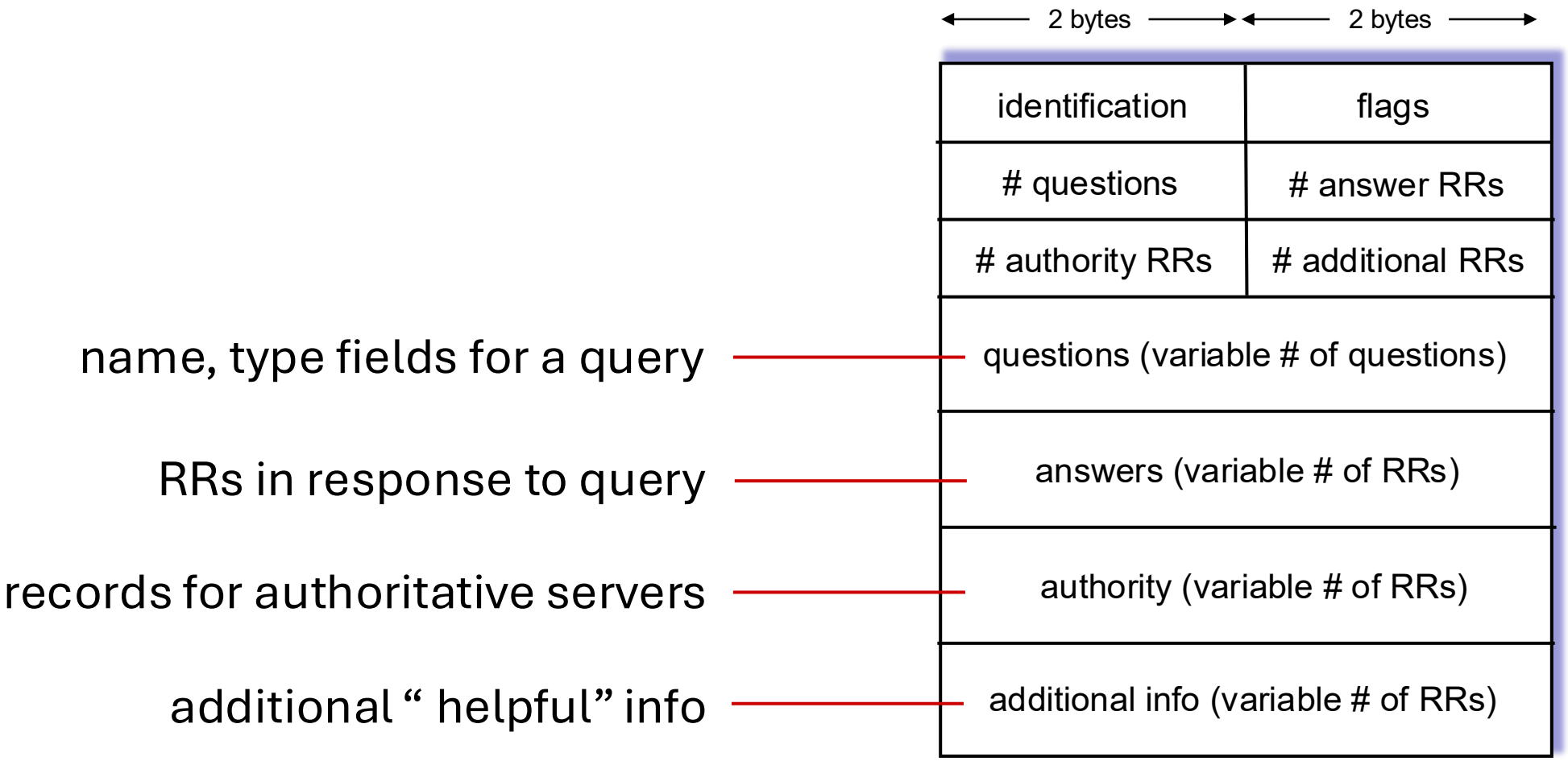
message header:

- identification:
  16 bit # for query, reply
  to query uses same identification
- flags:
  - query or reply (bit 0)
  - reply is authoritative (bit 5)
  - recursion desired (bit 7)
  - recursion available (bit 8)
  - response code (bits 12-15)

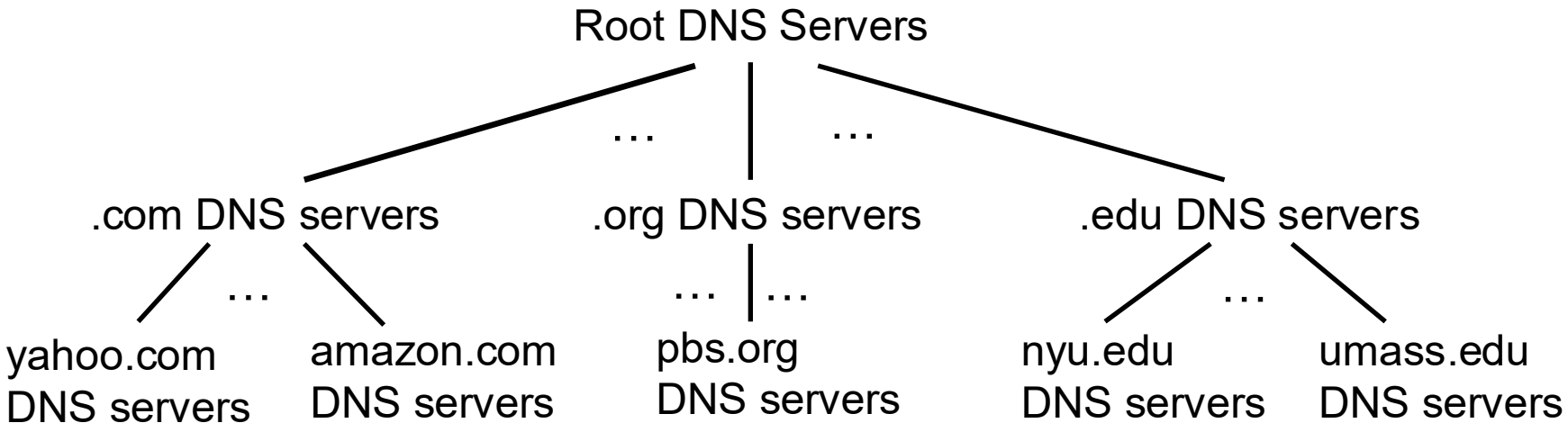


DNS protocol messages

- DNS *query* and *reply* messages, both have same *format:*



DNS: a distributed, hierarchical database



Root

Top Level Domain

Authoritative

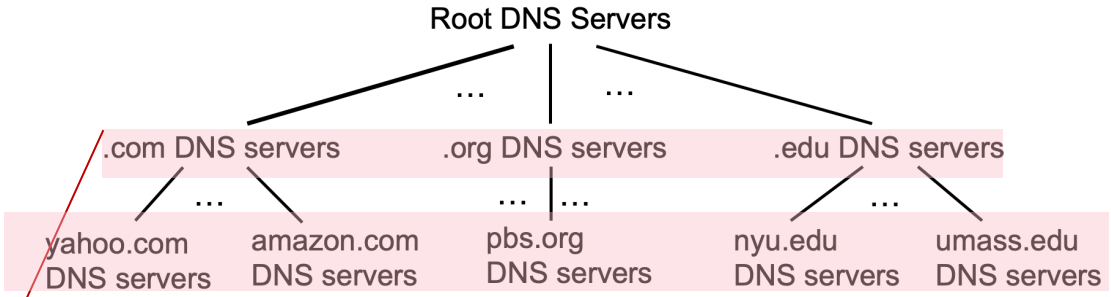
Distributed database:

- DNS data not contained in a single server
- Instead contained in millions of servers
- Each server contains a small part of the entire database

Authoritative and Top-Level Domain servers

authoritative DNS servers:

- organization's own DNS server(s), providing authoritative hostname to IP mappings for organization's named hosts
- can be maintained by organization or service provider on behalf of organization

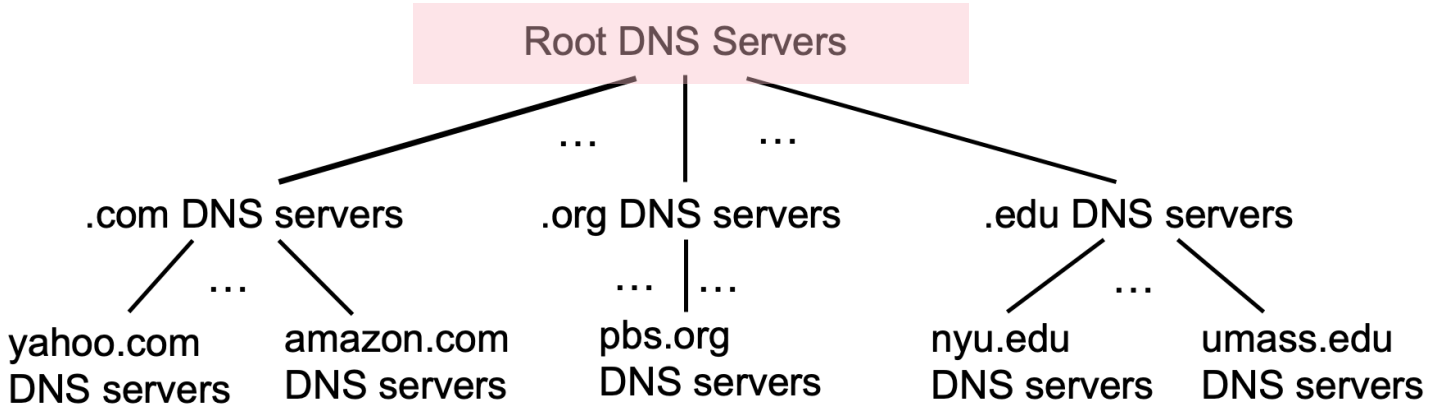


Top-Level Domain (TLD) servers:

- Each TLD server has data about authoritative servers in a particular top level domain (e.g. .com TLD server knows about all authoritative servers in .com TLD)
- Different TLDs responsible for .com, .org, .net, .edu, .aero, .jobs, .museums, and all top-level country domains, e.g.: .cn, .uk, .fr, .ca, .jp, .lk
- Network Solutions: .com, .net TLDs
- Educause: .edu TLD
- LK Domain Registry: .lk CCTLD

Root name servers

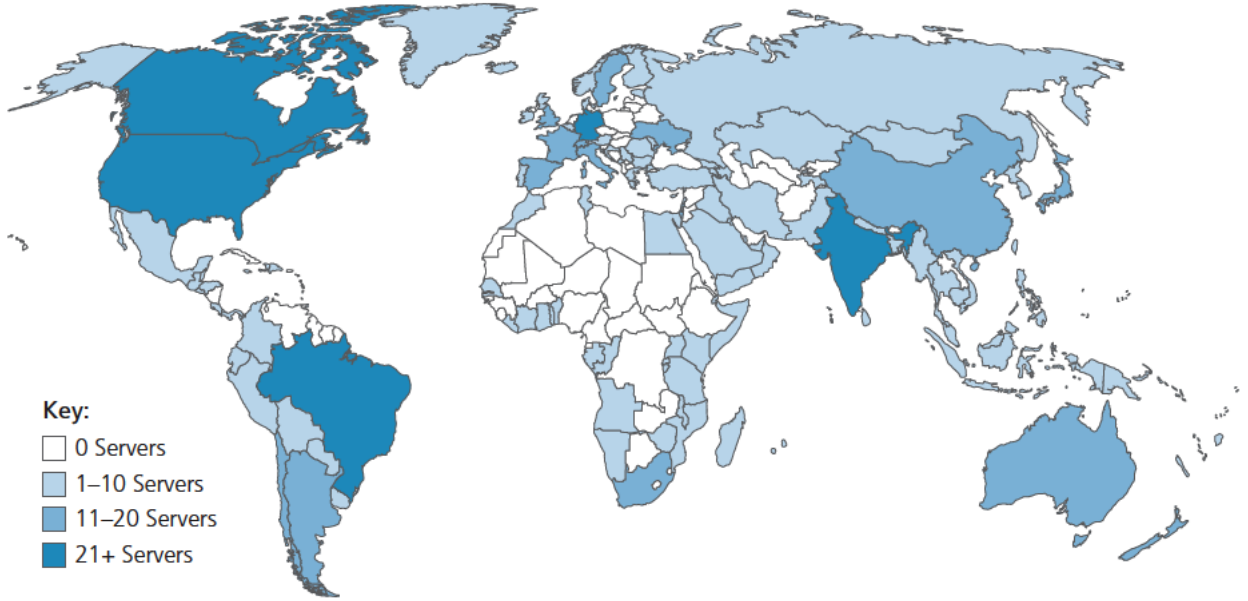
- Have data about all the TLD servers
- official, contact-of-last-resort by name servers that can not resolve name



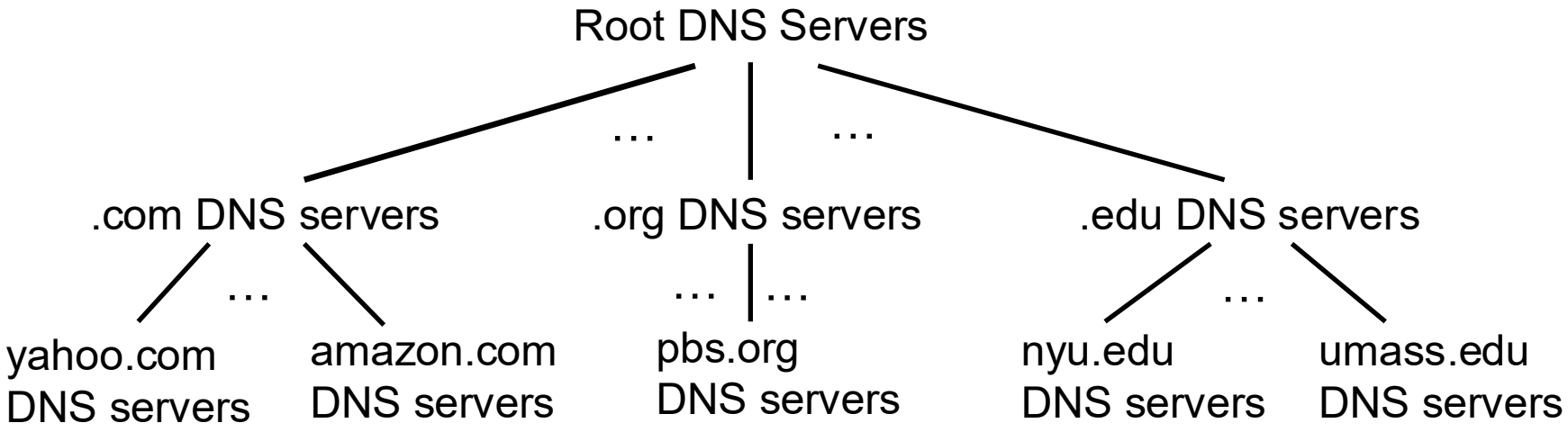
Root name servers

- official, contact-of-last-resort by name servers that can not resolve name
- *incredibly important* Internet function
  - Internet couldn't function without it!
  - DNSSEC – provides security (authentication, message integrity)
- ICANN (Internet Corporation for Assigned Names and Numbers) manages root DNS domain

13 logical root name "servers" worldwide each "server" replicated many times (~200 servers in US)



DNS resolution process



Root

Top Level Domain

Authoritative

Client wants IP address for www.amazon.com:

- client queries root server to find .com TLD nameserver
- client queries .com TLD nameserver to get amazon.com DNS server
- client queries amazon.com DNS server to get IP address for www.amazon.com

Local DNS servers

- Hosts usually do not carry out DNS resolution themselves
- Instead, they send the query to a *local* DNS server
  - Local DNS server returns a reply, answering the query
  - Every ISP/Enterprise/Home has a local DNS name server
  - To find yours:
    - MacOS: % scutil --dns
    - Windows: >ipconfig /all
- local DNS server doesn't belong to hierarchy
  - It is a server helping client hosts on local network in DNS resolution
  - Will usually query ROOT, TLD and Authoritative servers to get the answer the client host needs

Recursive name servers

- A recursive name server is a name server that will do the full work for replying to a query
  - If a client queries for the IP address of courseweb.sliit.lk, it will provide an IP address as the answer
  - To provide that final answer, the recursive server may contact other servers
- To make a recursive query, a client should set the recursion desired flag in the query
- A recursive server will provide the answer, and set the recursion available flag in the response
- Local DNS servers and public DNS servers like 8.8.8.8 and 1.1.1.1 are recursive servers

Non-Recursive name servers

- A non-recursive name server (also called authoritative name server) will NOT query other name servers to resolve a query
- Usually, a non-recursive name server will refer the client to another name server (this is called a referral)
  - "I can't provide the answer you are looking for, but contact this server"
- The response of a non-recursive server will have recursion available flag set to 0
- Root, TLD and Authoritative name servers are non-recursive name servers

DNS resolution: Iteration vs recursion

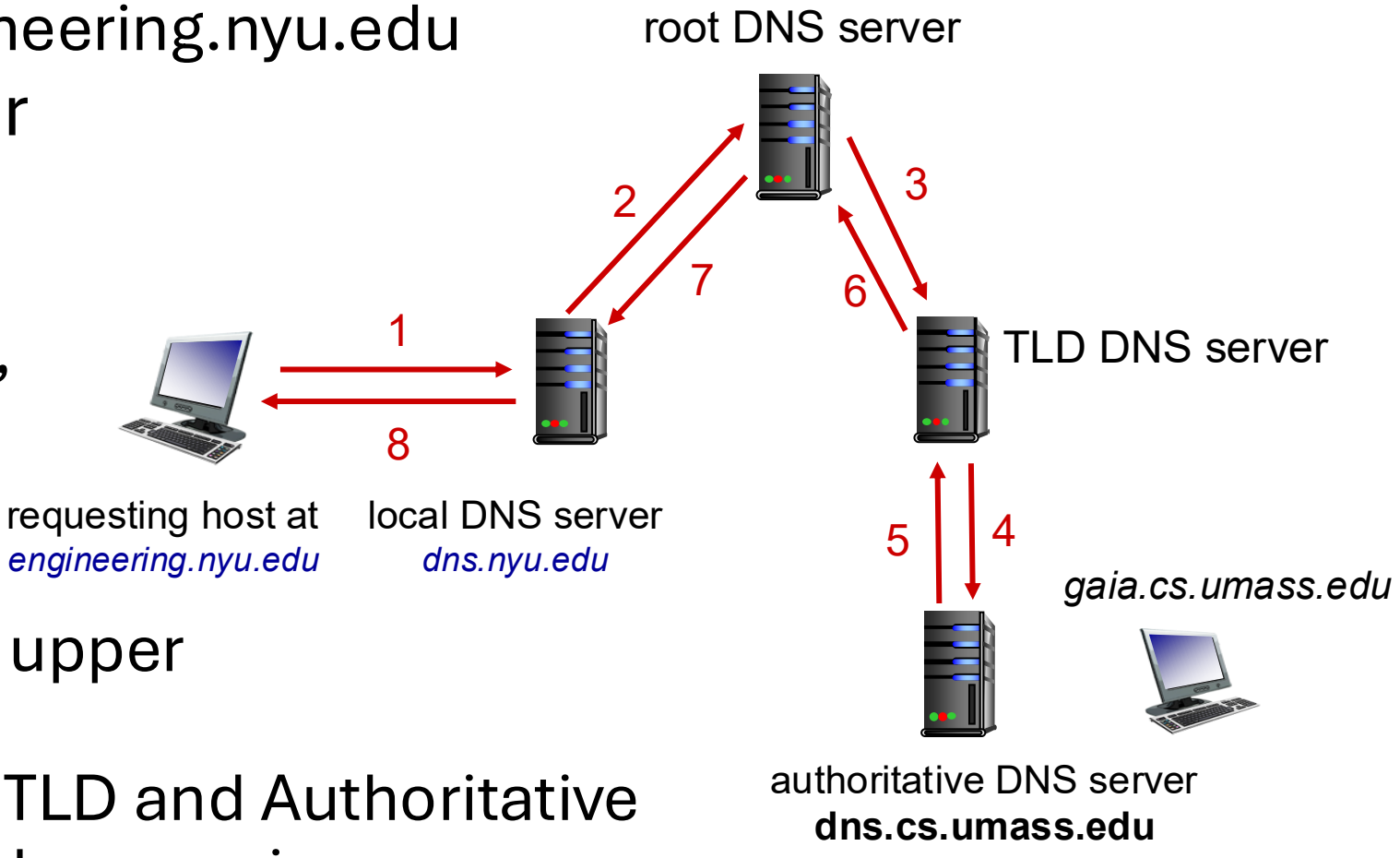
- A DNS query may request recursion
  - recursion desired flag in the query is set
- But the queried server may refuse to provide recursion, and instead provide the details of the nameserver(s) to query next
  - recursion available flag in the reply is NOT set
- In practice, the queries typically follow the pattern in slide 8-32
  - Query from client host to local DNS server has recursion desired flag set
  - Local DNS server will accept request for recursion, and do all queries needed (on behalf of the requesting host) iteratively
  - When it has the final answer, it provides the answer to the requesting host
  - ROOT , TLD and Authoritative servers do not honor requests for recursion

DNS name resolution: recursive resolution

Example: host at engineering.nyu.edu wants IP address for gaia.cs.umass.edu

Recursive resolution:

- What happens if Root, TLD and Authoritative servers provided recursion?
- Creates heavy load at upper levels of hierarchy
- For this reason, Root, TLD and Authoritative servers DO NOT provide recursion

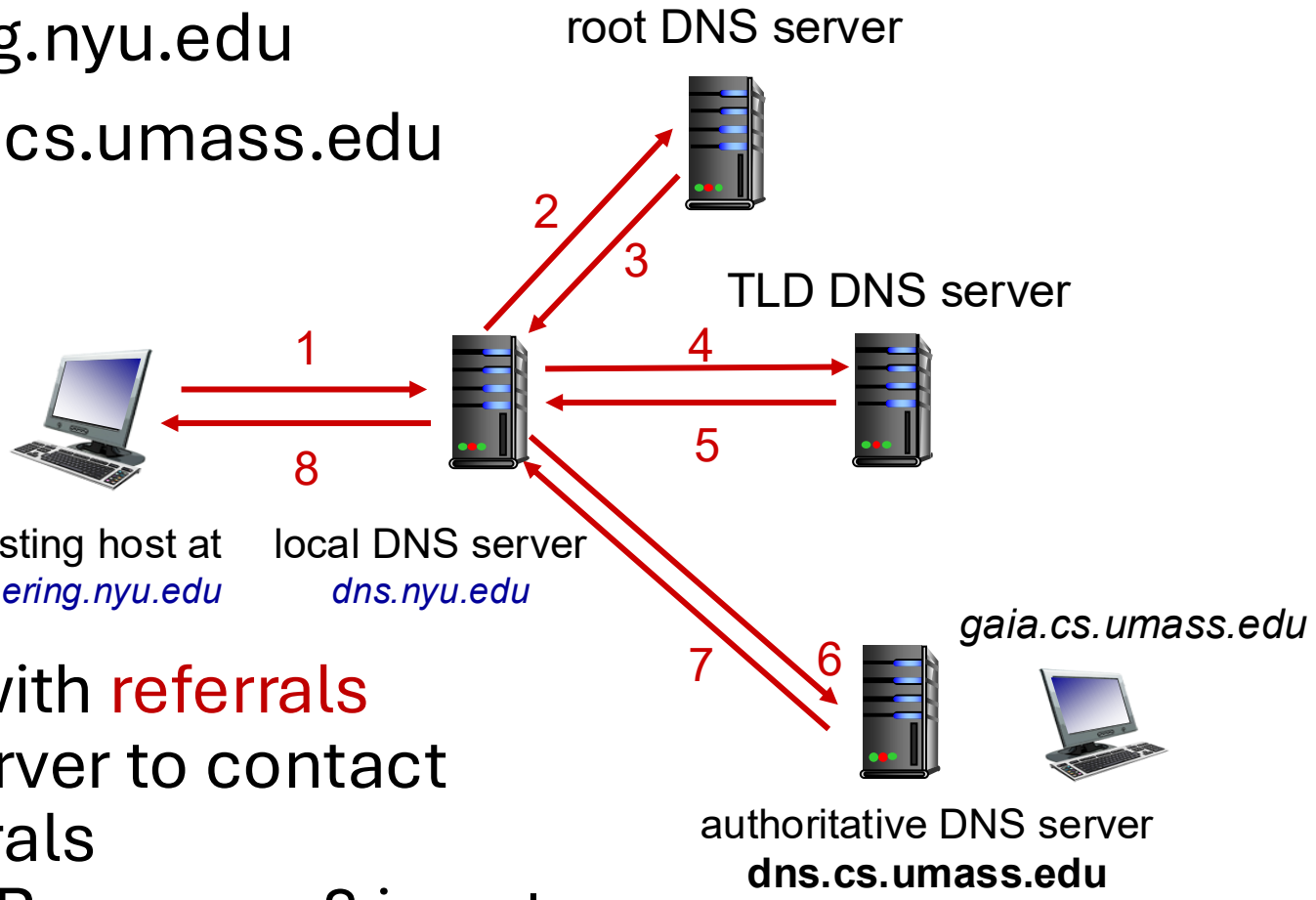


DNS name resolution: iterated resolution

Example: host at engineering.nyu.edu wants IP address for gaia.cs.umass.edu

Iterated resolution:

- Because Root, TLD and authoritative servers do not provide recursion, Local DNS server resolves query iteratively
- Root and TLD servers reply with referrals name/address of another server to contact
- Responses 3 and 5 are referrals
- Response 7 is authoritative, Response 8 is not



Caching DNS Information

- Local name *caches* the mappings that it learns
- Next time a client queries for same information, it can *immediately* return a cached mapping in response
  - caching improves response time
  - cache entries timeout (disappear) after some time (TTL)
  - TLD servers typically cached in local name servers
- cached entries may be *out-of-date*
  - if named host changes IP address, may not be known Internet-wide until TTLs of all cached entries expire!
  - best-effort name-to-address translation!

DNS records

DNS: distributed database storing resource records (RR)

RR format: (`name, value, type, ttl`)

type=A

- `name` is hostname
- `value` is IP address

type=NS

- `name` is domain (e.g., foo.com)
- `value` is hostname of authoritative name server for this domain

type=CNAME

- `name` is alias name for some "canonical" (the real) name
- www.ibm.com is really servereast.backup2.ibm.com
- `value` is canonical name

type=MX

- `value` is name of SMTP mail server associated with `name`

DNS query types

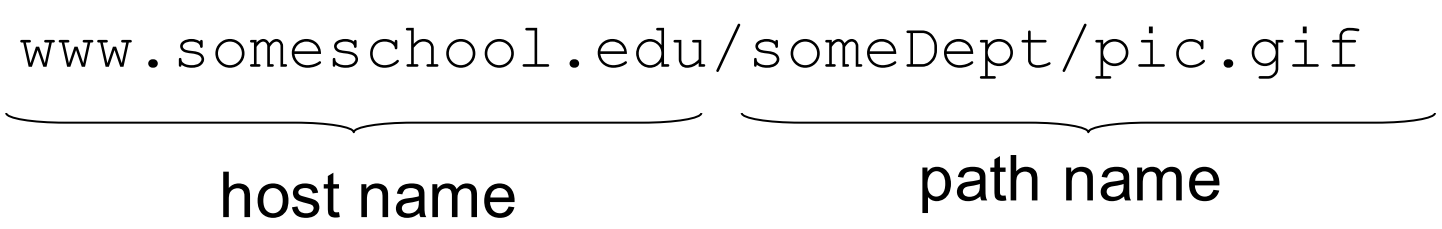
- When making a DNS query, client has to specify type of query
- Query type A: to find the IPv4 address of the domain in query
  - domain=courseweb.sliit.lk, query type=A
- Query type AAAA: to find the IPv6 address of the domain in query
  - domain=wikipedia.org, query type=AAAA
- Query type MX: to find a server which will accept mail for domain
  - domain=yahoo.com, query type=MX
- Query type NS: to find a name server authoritative for domain
  - domain=amazon.com, query type=NS
- Query type ANY: provide all available RRs for the domain
  - domain=sliit.lk, query type=ANY

Lesson outline

- Internet basics
- Application layer: overview
- DNS
- HTTP

Web and HTTP

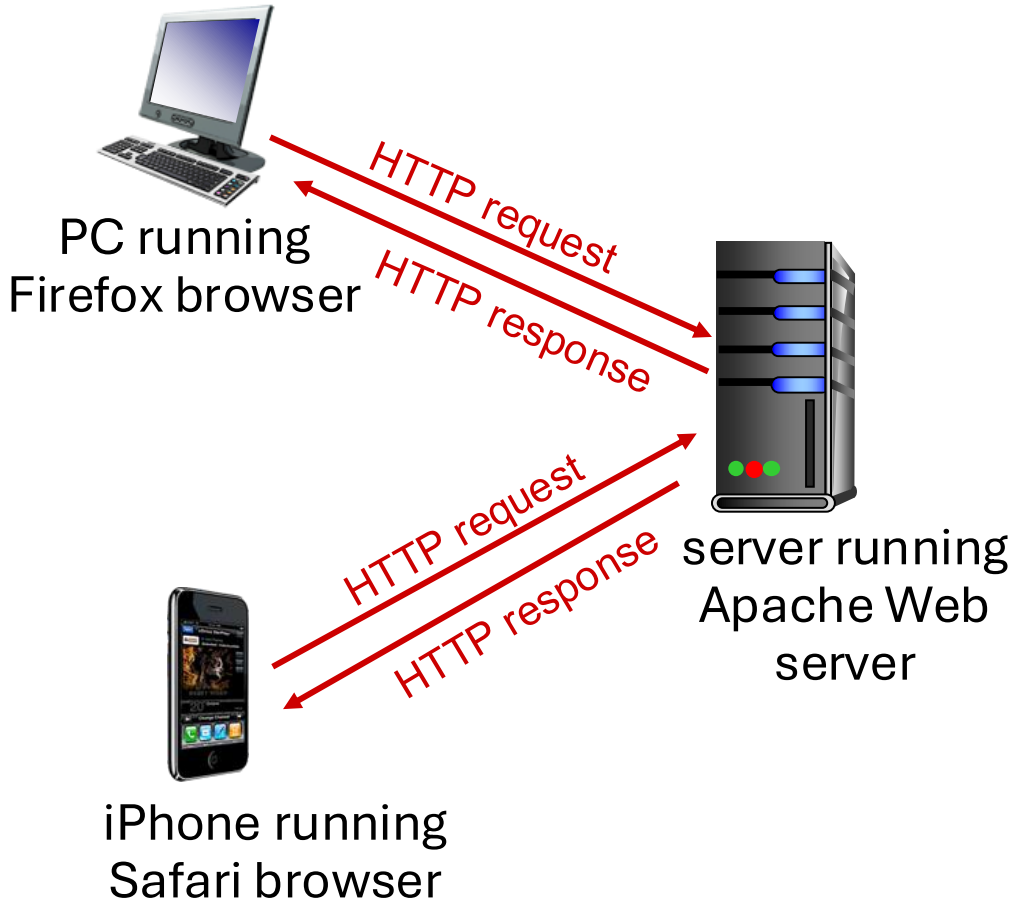
- web page consists of objects, each of which can be coming from different Web servers
- object can be HTML file, JPEG image, Java applet, audio file,…
- web page consists of base HTML-file which includes several referenced objects
- each object in page addressable by a URL, e.g.,



HTTP overview

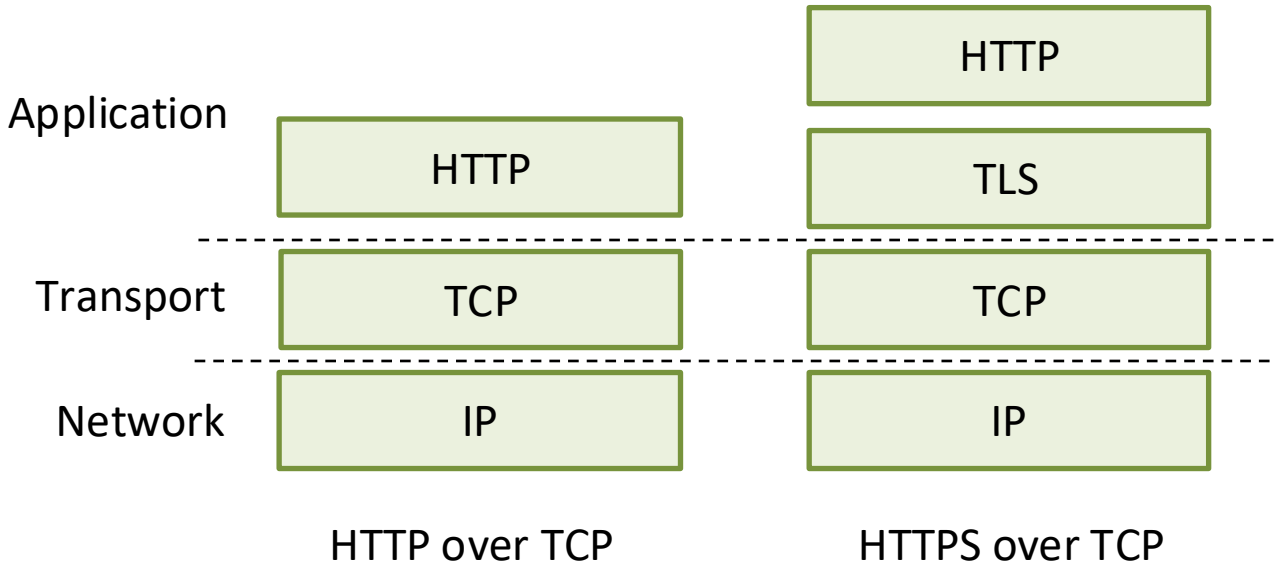
HTTP: hypertext transfer protocol

- Web's application-layer protocol
- client/server model:
  - client: browser that requests, receives, (using HTTP protocol) and "displays" Web objects
  - server: Web server sends (using HTTP protocol) objects in response to requests



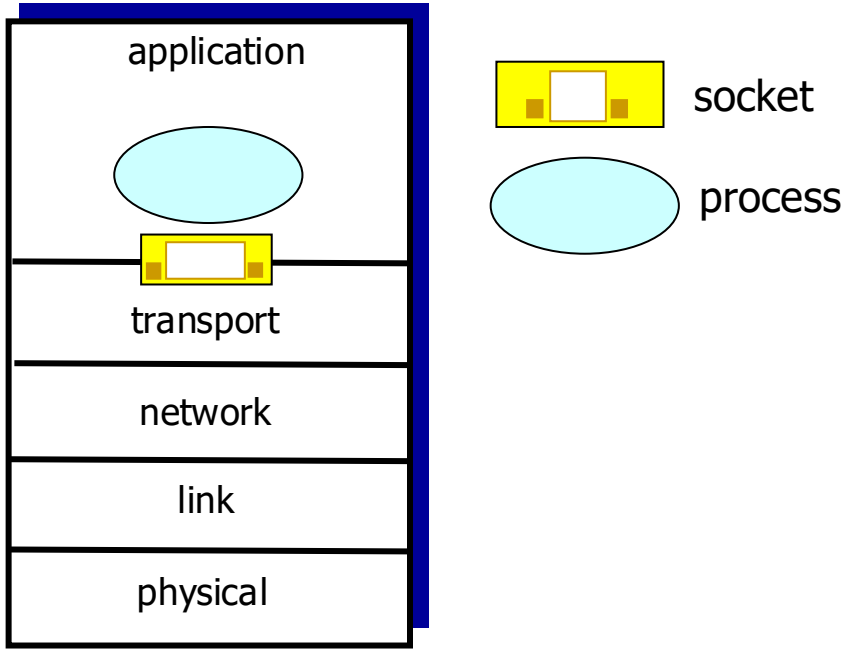
HTTPS

- HTTPS means secure HTTP
- But HTTPS is not a separate protocol
- Security (encryption) provided by TLS (an application layer protocol sitting in between HTTP and TCP



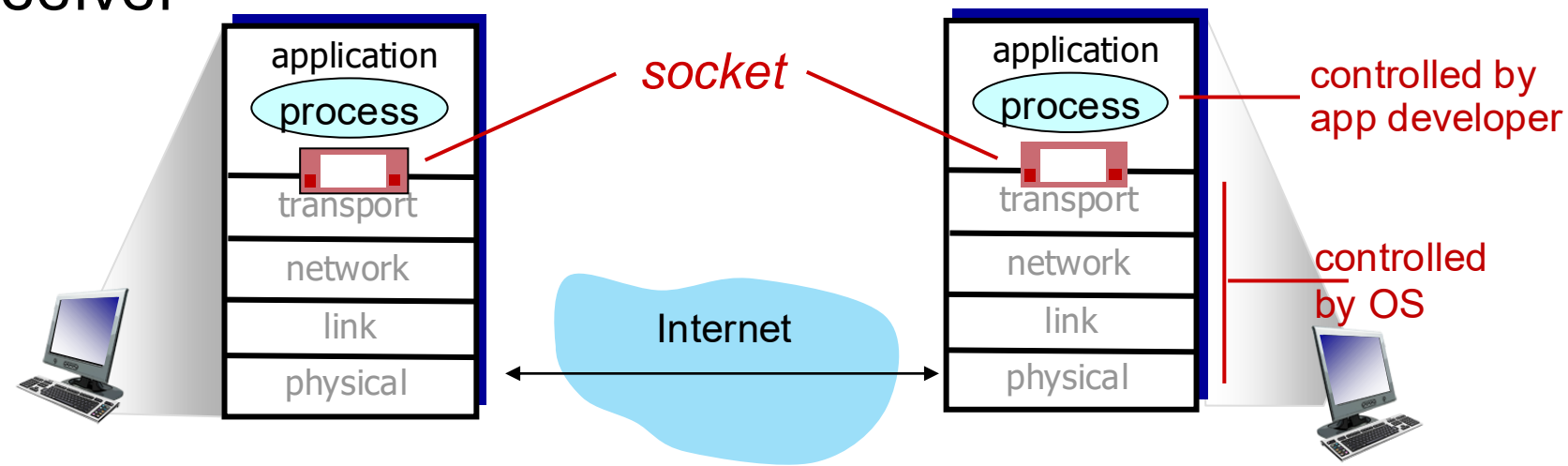
Sockets

- A process in the Application Layer connects to the Transport Layer via a socket
  - Process = running app / part of a running app
- A socket is an endpoint for sending or receiving data across a network
- Socket created by application layer process
  - socket is connected to that process
- A socket is identified by the protocol, IP address and port number
  - Plus remote IP and port number in TCP
  - More about this in Lesson 7 (Transport Layer)



Sockets (ctd)

- A socket is like a door
  - process sends/receives messages to/from its socket
  - sending process pushes message out through the door
  - sending process relies on transport layer on other side of door to deliver message to socket at receiving process
  - communication involves two sockets: one at sender and one at receiver



HTTP overview (continued)

HTTP uses TCP:

- client initiates TCP connection (creates socket) to server, port 80
- server accepts TCP connection from client
- HTTP messages (application-layer PDUs) exchanged between browser (HTTP client) and Web server (HTTP server)
- TCP connection closed

HTTP is "stateless"

- server maintains no information about past client requests

aside

protocols that maintain "state" are complex!

- past history (state) must be maintained
- if server/client crashes, their views of "state" may be inconsistent with the other

HTTP connections: two types

Non-persistent HTTP

1. TCP connection opened
2. at most one object sent over TCP connection
3. TCP connection closed

downloading multiple objects requires multiple connections

Persistent HTTP

- TCP connection opened to a server
- multiple objects can be sent over single TCP connection between client, and that server
- TCP connection closed

Non-persistent HTTP: example

User enters URL: **`www.someSchool.edu/someDepartment/home.index`**
(containing text, references to 10 jpeg images)



1a. HTTP client initiates TCP connection to HTTP server (process) at www.someSchool.edu on port 80



1b. HTTP server at host www.someSchool.edu waiting for TCP connection at port 80 "accepts" connection, notifying client

2. HTTP client sends HTTP request message (containing URL) into TCP connection socket. Message indicates that client wants object someDepartment/home.index

3. HTTP server receives request message, forms response message containing requested object, and sends message into its socket

time

Non-persistent HTTP: example (cont.)

User enters URL: **`www.someSchool.edu/someDepartment/home.index`**
(containing text, references to 10 jpeg images)





4. HTTP server closes TCP connection.

5. HTTP client receives response message containing html file, displays html. Parsing html file, finds 10 referenced jpeg objects

6. Steps 1-5 repeated for each of 10 jpeg objects

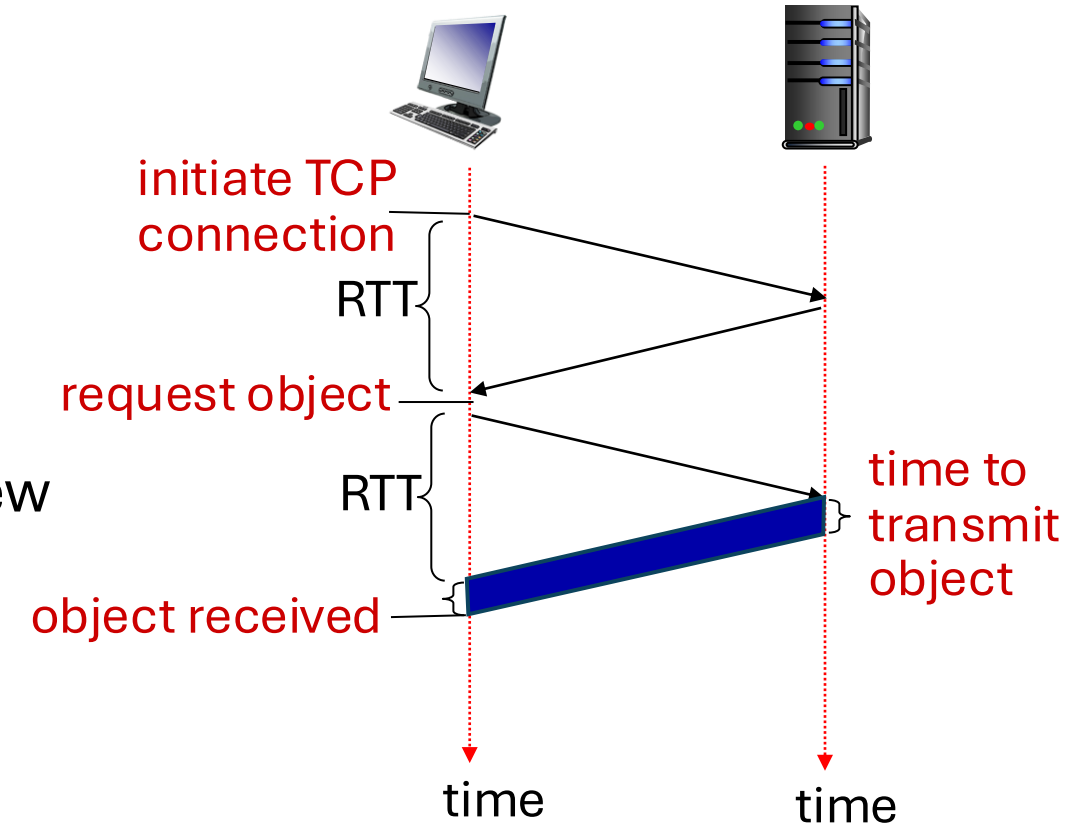
time

Non-persistent HTTP: response time

RTT (definition): time for a small packet to travel from client to server and back

HTTP response time (per object):

- one RTT to initiate TCP connection
- one RTT for HTTP request and first few bytes of HTTP response to return
- object transmission time



Non-persistent HTTP response time = 2RTT+ object transmission time

Persistent HTTP (HTTP 1.1)

Non-persistent HTTP issues

- requires 2 RTTs per object
- OS overhead for each TCP connection
- browsers often open multiple parallel TCP connections to fetch referenced objects in parallel

Persistent HTTP (HTTP1.1)

- server leaves connection open after sending response
- subsequent HTTP messages between same client/server sent over open connection
- client sends requests as soon as it encounters a referenced object
- as little as one RTT for all the referenced objects (cutting response time in half)

HTTP 1.1

Key goal: decreasing delay in multi-object HTTP requests

HTTP1.1: introduced multiple, pipelined GETs over single TCP connection

- server responds in-order (FCFS: first-come-first-served scheduling) to GET requests
- with FCFS, small object may have to wait for transmission (head-of-line (HOL) blocking) behind large object(s)
- loss recovery (retransmitting lost TCP segments) stalls object transmission

HTTP/2

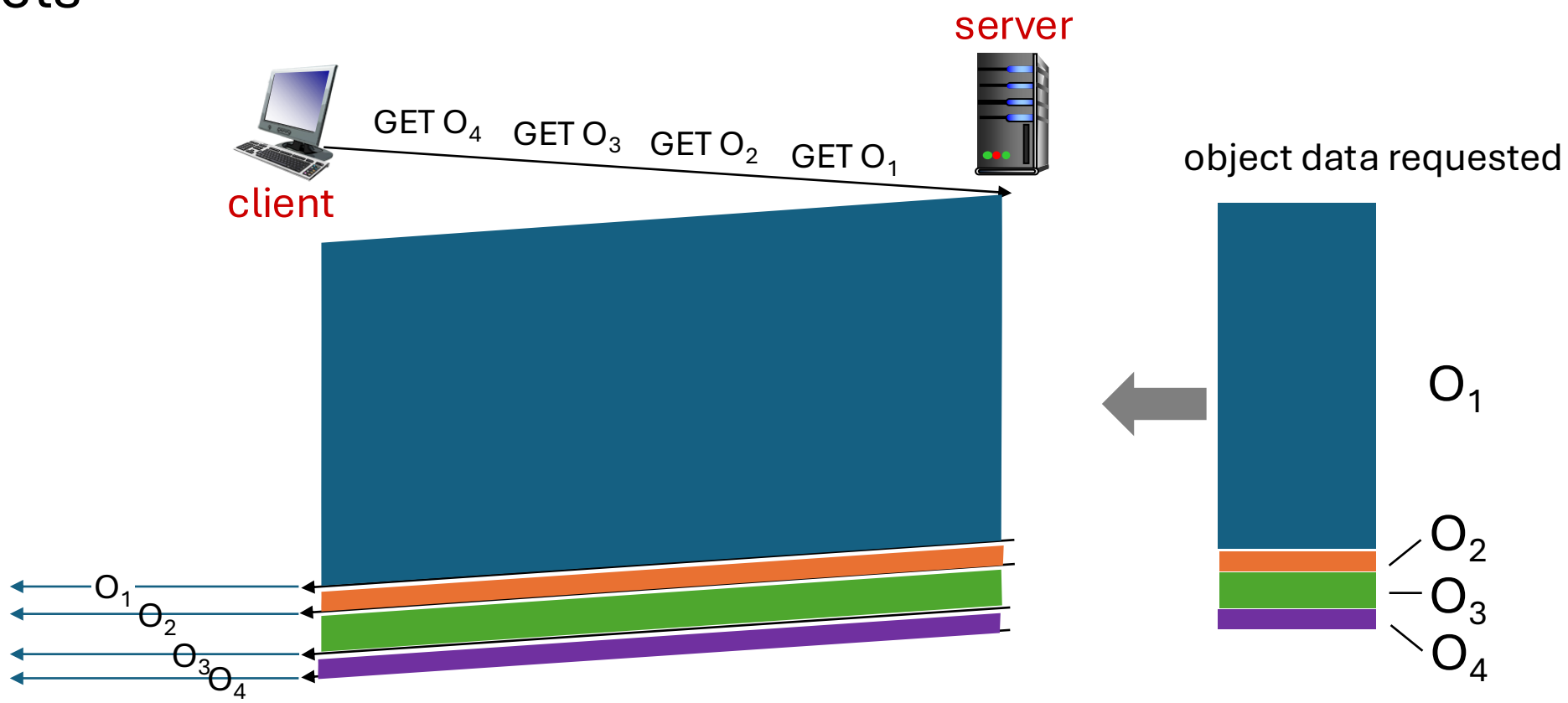
Key goal: decreasing delay in multi-object HTTP requests

HTTP/2: [RFC 7540, 2015] increased flexibility at server in sending objects to client:

- methods, status codes, most header fields unchanged from HTTP 1.1
- transmission order of requested objects based on client-specified object priority (not necessarily FCFS)
- push unrequested objects to client
- divide objects into frames, schedule frames to mitigate HOL blocking

HTTP/2: mitigating HOL blocking

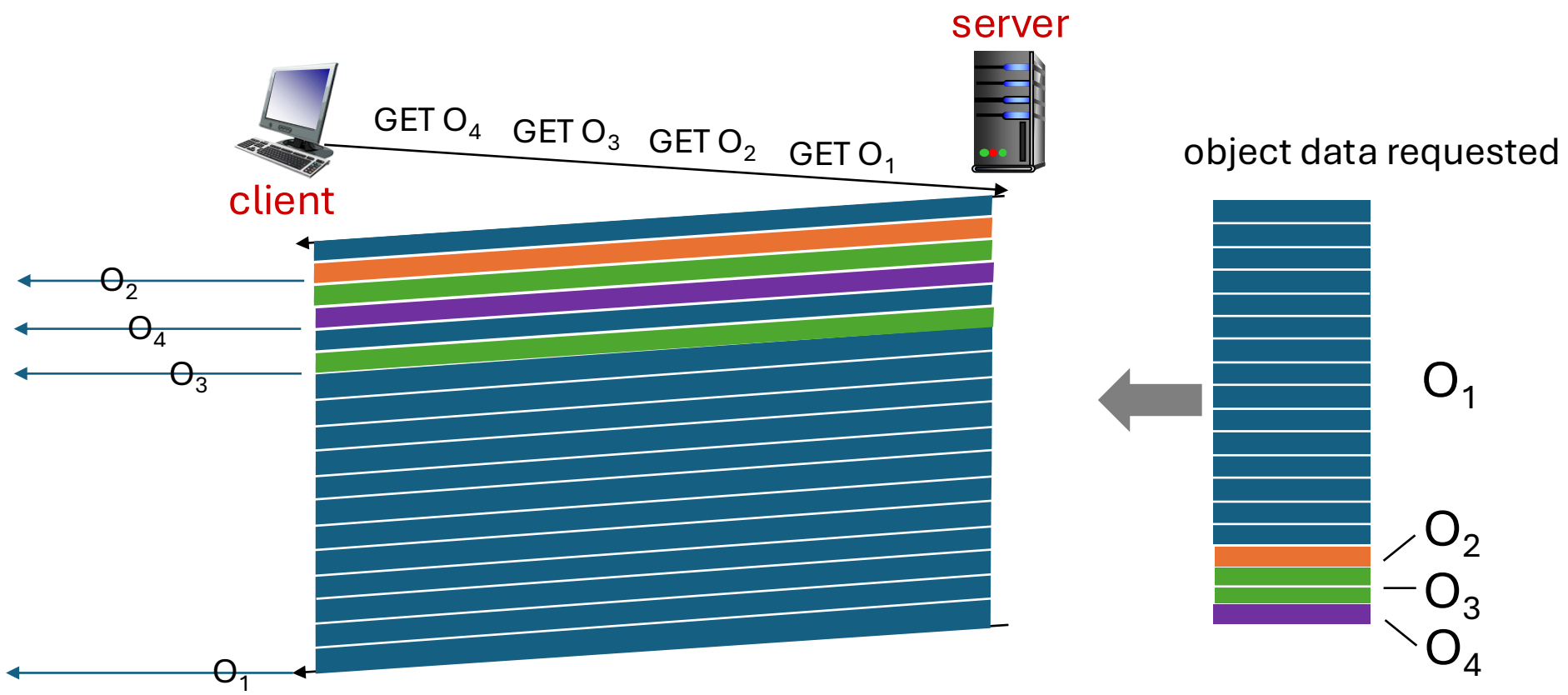
HTTP 1.1: client requests 1 large object (e.g., video file) and 3 smaller objects



objects delivered in order requested: O_2, O_3, O_4 wait behind O_1

HTTP/2: mitigating HOL blocking

HTTP/2: objects divided into frames, frame transmission interleaved



O_2, O_3, O_4 delivered quickly, O_1 slightly delayed

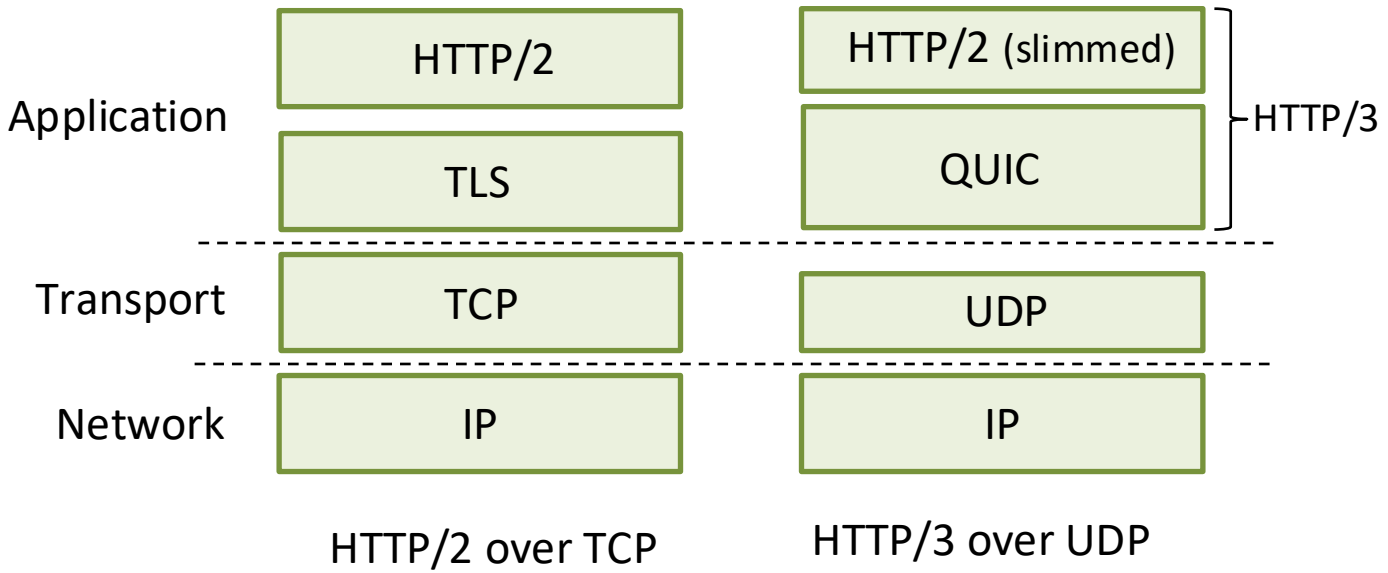
HTTP/2 to HTTP/3

HTTP/2 over single TCP connection means:

- recovery from packet loss still stalls all object transmissions
  - as in HTTP 1.1, browsers have incentive to open multiple parallel TCP connections to reduce stalling, increase overall throughput
- no security over vanilla TCP connection
- HTTP/3: adds security, per object error- and congestion-control (more pipelining) over UDP

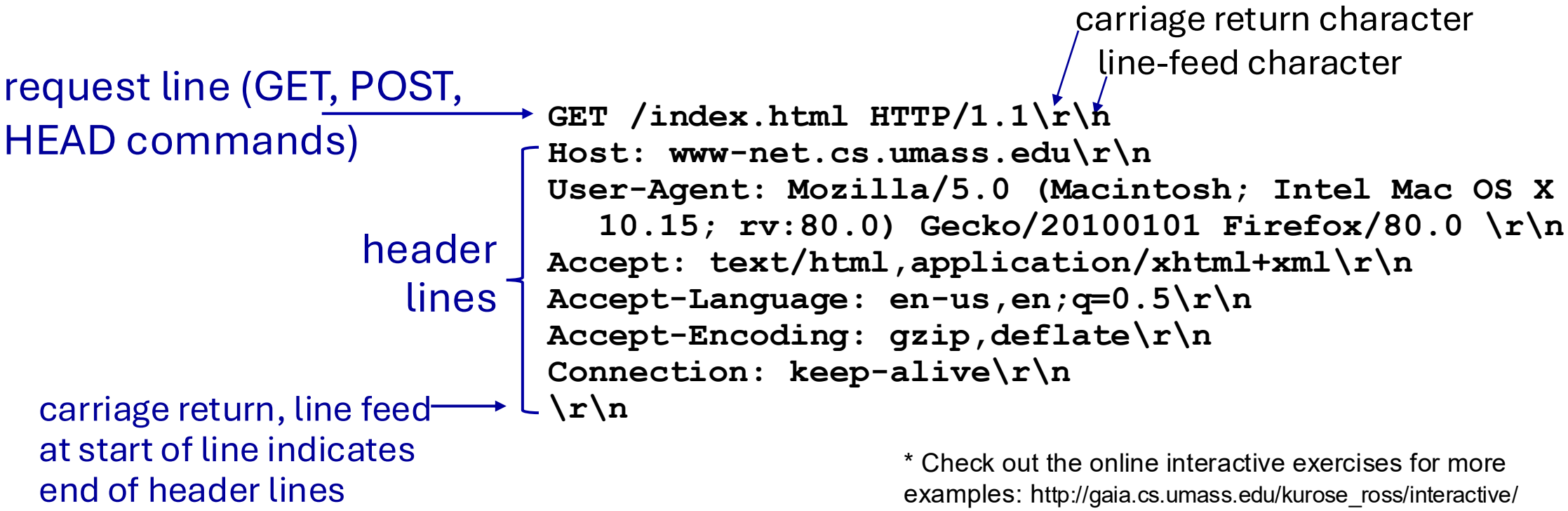
QUIC: Quick UDP Internet Connections

- application-layer protocol, on top of UDP
  - increase performance of HTTP
  - deployed on many Google servers, apps (Chrome, mobile YouTube app)



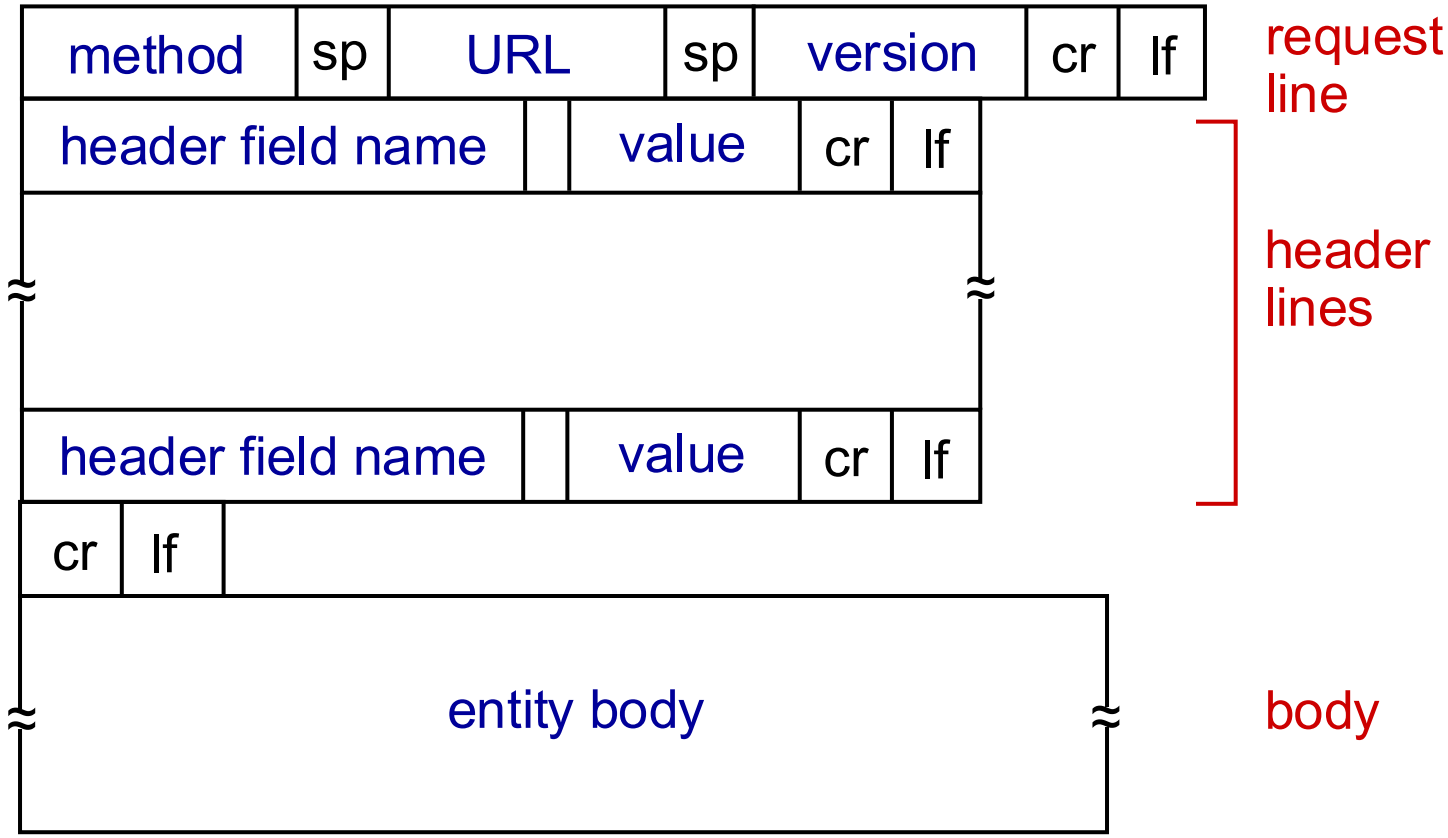
HTTP request message

- two types of HTTP messages: request, response
- HTTP request message:
  - ASCII (human-readable format)



* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

HTTP request message: general format



Other HTTP request messages

POST method:

- web page often includes form input
- user input sent from client to server in entity body of HTTP POST request message

GET method (for sending data to server):

- include user data in URL field of HTTP GET request message (following a '?'):

```
www.somesite.com/animalsearch?monkeys&banana
```

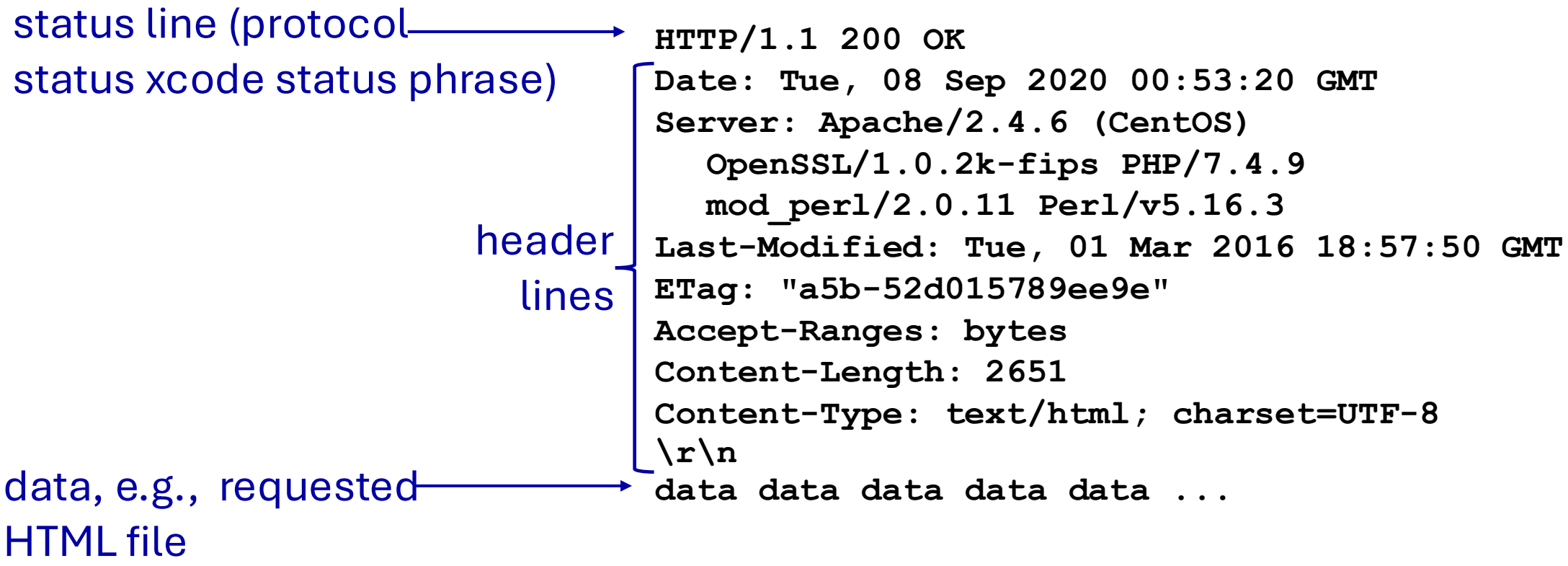
HEAD method:

- requests headers (only) that would be returned if specified URL were requested with an HTTP GET method.

PUT method:

- uploads new file (object) to server
- completely replaces file that exists at specified URL with content in entity body of POST HTTP request message

HTTP response message



* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

HTTP response status codes

- status code appears in first line in server-to-client response message
- some sample codes:

200 OK

- request succeeded, requested object later in this message

301 Moved Permanently

- requested object moved, new location specified later in this message (in Location: field)

400 Bad Request

- request msg not understood by server

404 Not Found

- requested object not found on this server

505 HTTP Version Not Supported

Maintaining user/server state: cookies

Web sites and client browser use cookies to maintain some state between transactions

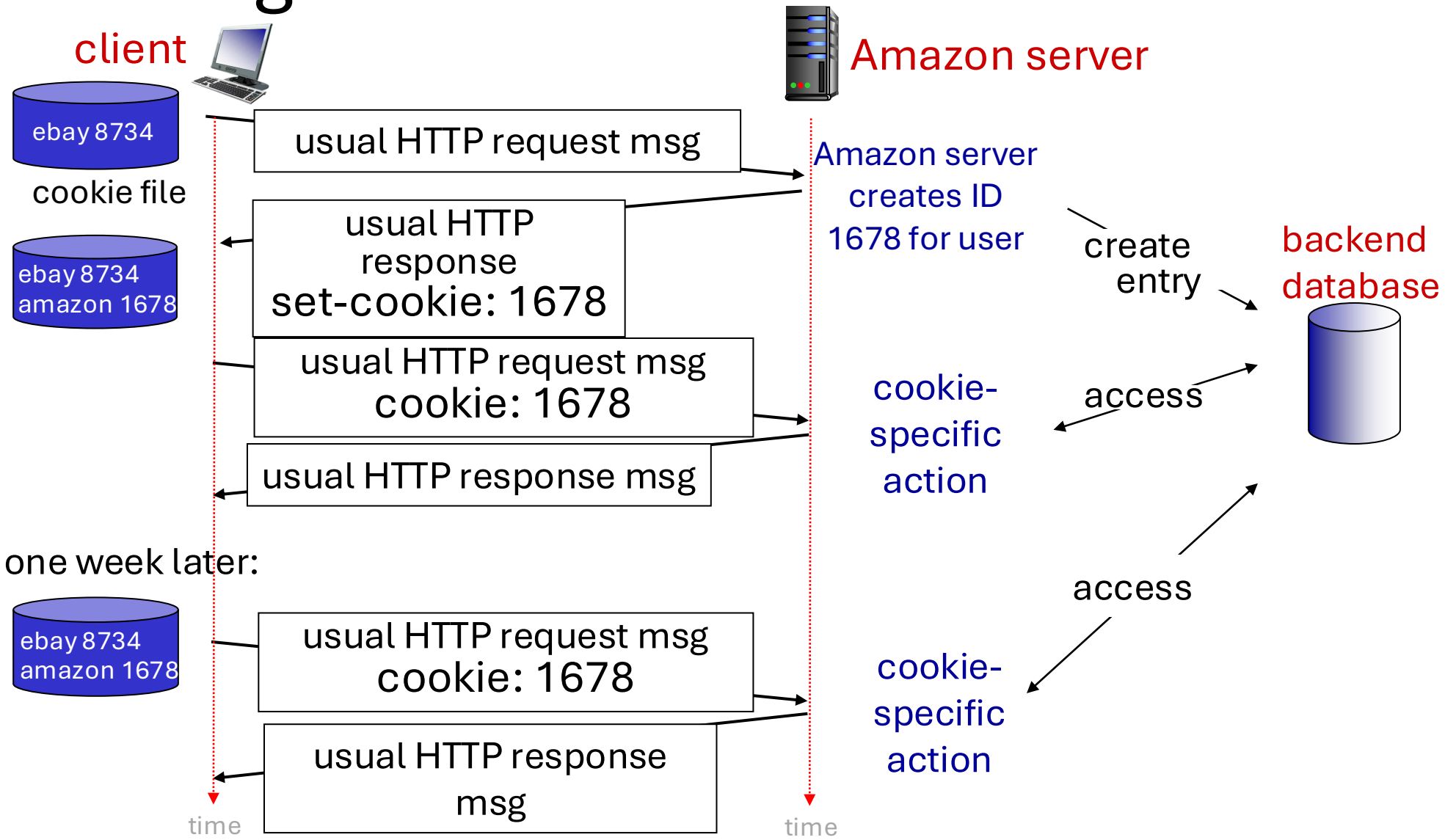
four components:

1) cookie header line of HTTP response message
2) cookie header line in next HTTP request message
3) cookie file kept on user's host, managed by user's browser
4) back-end database at Web site

Example:

- Susan uses browser on laptop, visits specific e-commerce site for first time
- when initial HTTP requests arrives at site, site creates:
  - unique ID (aka "cookie")
  - entry in backend database for ID
- subsequent HTTP requests from Susan to this site will contain cookie ID value, allowing site to "identify" Susan

Maintaining user/server state: cookies



HTTP cookies: comments

Cookies can be used for:

- authorization
- shopping carts
- recommendations
- user session state (Web e-mail)

Challenge: How to keep state?

- at protocol endpoints: maintain state at sender/receiver over multiple transactions
- in messages: cookies in HTTP messages carry state

Cookies and privacy:

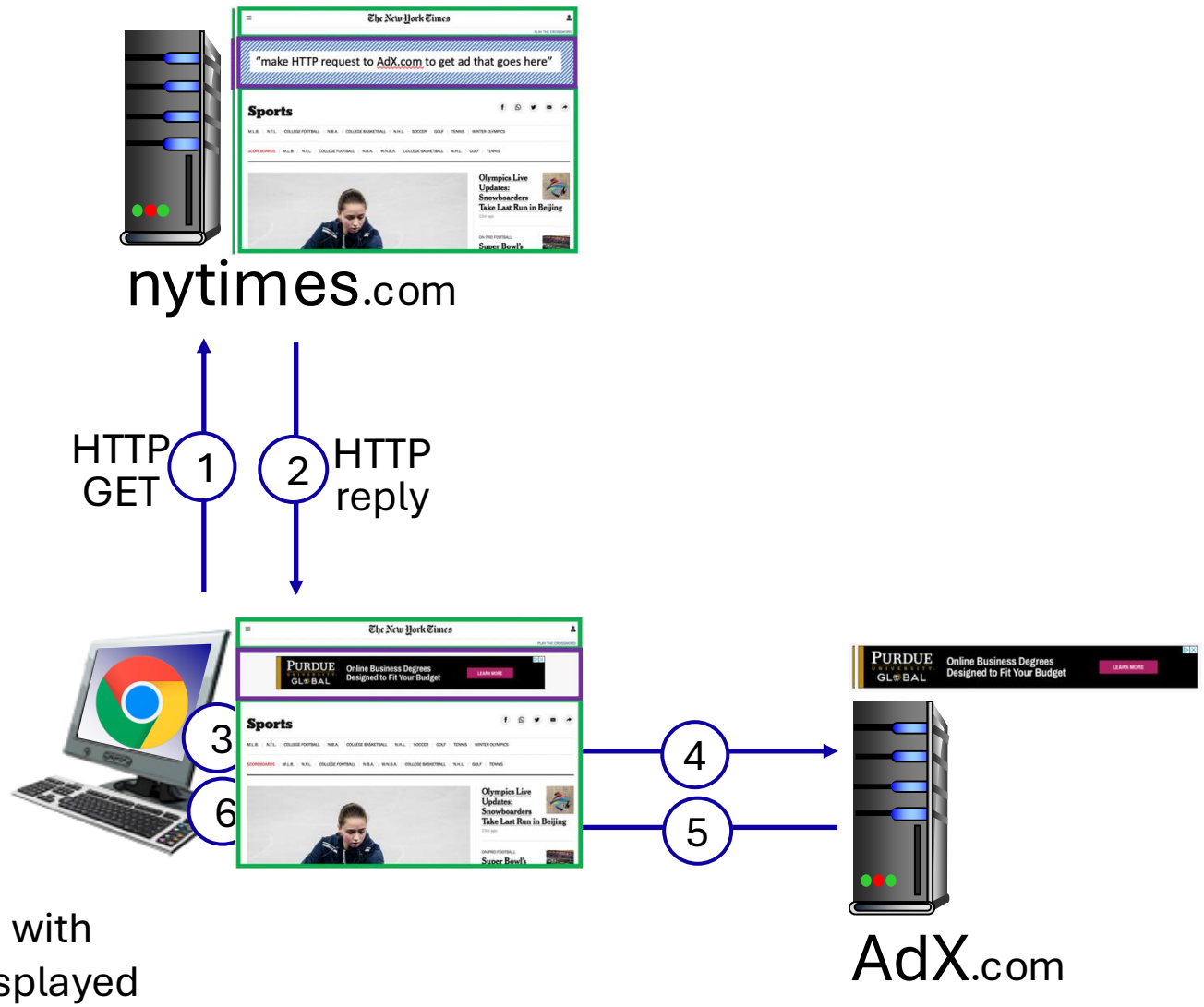
- cookies permit sites to learn a lot about you on their site.
- third party persistent cookies (tracking cookies) allow common identity (cookie value) to be tracked across multiple web sites

Example: displaying a NY Times web page

1 2 GET base html file from nytimes.com

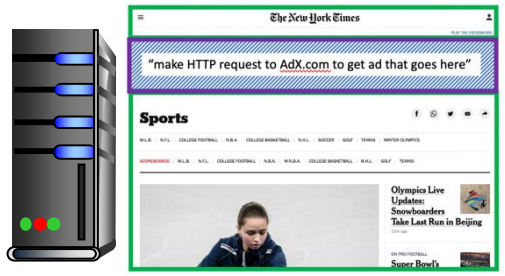
4 5 fetch ad from AdX.com

7 display composed page



NY times page with embedded ad displayed

Cookies: tracking a user's browsing behavior



nytimes.com (sports)

1634: sports, 2/15/22

"first party" cookie – from website you chose to visit

HTTP GET

HTTP reply
Set cookie: 1634



"third party" cookie – from website you did not choose to visit

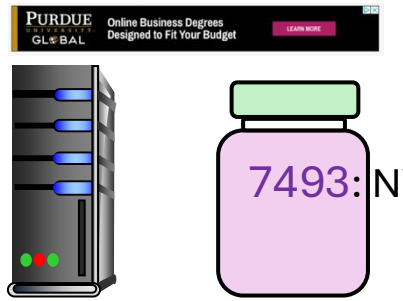
NY Times: 1634
AdX: 7493

HTTP GET
Referrer: NY Times Sports

4

5

HTTP reply
Set cookie: 7493



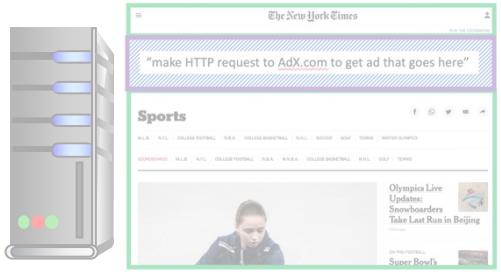
7493: NY Times sports, 2/15/22

AdX.com

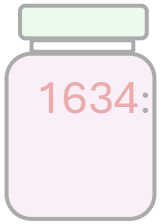
Cookies: tracking a user's browsing behavior



socks.com



nytimes.com



HTTP reply 2

HTTP GET 1

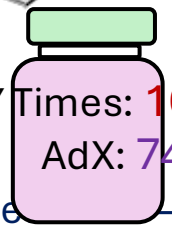


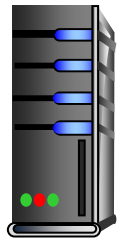
HTTP GET
Referrer: socks.com, cookie: 7493

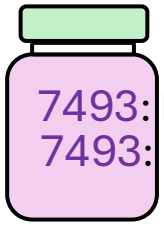
4

5

HTTP reply
Set cookie: 7493







AdX.com

AdX:

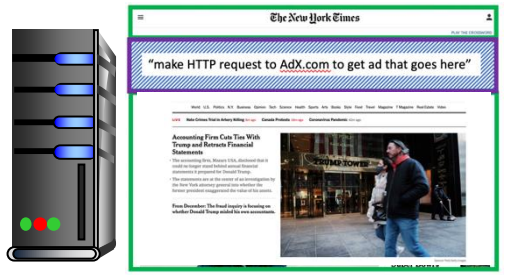
- tracks my web browsing over sites with AdX ads
- can return targeted ads based on browsing history

Cookies: tracking a user's browsing behavior

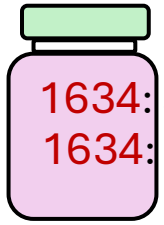
(one day later)



socks.com



nytimes.com (arts)



HTTP GET
cookie: 1634

HTTP reply
Set cookie: 1634



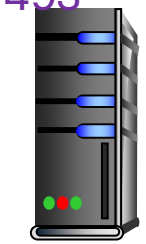
HTTP GET
Referrer: nytimes.com, cookie: 7493

4

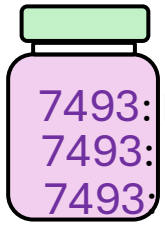
5

HTTP reply
Set cookie: 7493
Returned ad for socks!





AdX.com



Cookies: tracking a user's browsing behavior

Cookies can be used to:

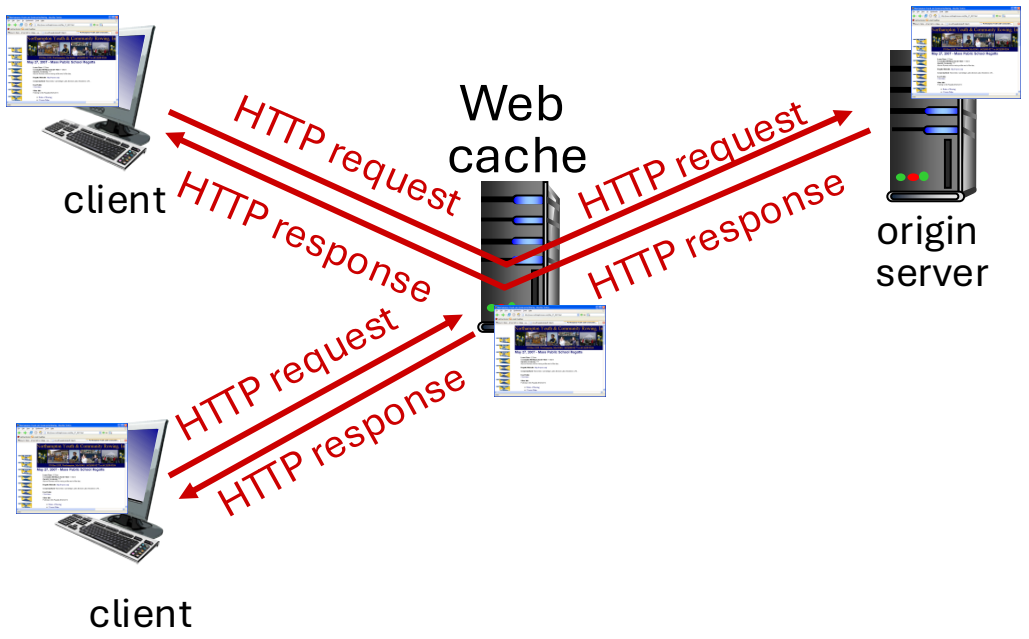
- track user behavior on a given website (first party cookies)
- track user behavior across multiple websites (third party cookies) without user ever choosing to visit tracker site (!)
- tracking may be *invisible* to user:
  - rather than displayed ad triggering HTTP GET to tracker, could be an invisible link

third party tracking via cookies:

- disabled by default in many browsers
- can be disabled in other browsers

Web caches

- Goal: satisfy client requests without involving origin server
- user configures browser to point to a (local) Web cache server
- browser sends all HTTP requests to cache
  - if object in cache: cache returns object to client
  - else cache requests object from origin server, caches received object, then returns object to client



Web caches (aka proxy servers)

- Web cache acts as both client and server
  - server for original requesting client
  - client to origin server
- server tells cache about object's allowable caching in response header:

```
Cache-control: max-age=50
Cache-control: no-cache
```

Why Web caching?

- reduce response time for client request
  - cache is closer to client
- reduce traffic on an institution's access link
- Internet is dense with caches
  - enables "poor" content providers to more effectively deliver content

Review questions

- What are the types of DNS servers?
- Explain the queries and responses in an
  - iterative query
  - recursive query
- List 4 different types of DNS records.
- What are the benefits of DNS caching?
- What is the difference between persistent HTTP and non-persistent HTTP?
- What are third party cookies used in web access?

References

- Chapter 2 - The Application Layer
  Chapter 4 - The Network Layer: Data Plane
  James F. Kurose and Keith W. Ross, Computer Networking – A Top-Down Approach, (8th Edition), Pearson, 2020

Lecture slides prepared by

Nimal Ratnayake
Dept of Computer Systems Engineering
SLIIT

First version: 2024 July
Revision 1: 2025 February
Revision 2: 2025 July
Revision 3: 2026 July